

Linux — Administration avancée

LUX-ADMA — 3 jours

M2I Formation

Support complet — 6 modules

Programme des trois jours

Jour	Module
J1 matin	1. Architecture système Linux et compilation du noyau
J1 après-midi	2. Modules de noyau, /proc, /sys et dépannage matériel
J2 matin	3. Stockage avancé : LVM et BTRFS
J2 après-midi	4. Séquence d'amorçage et gestion de l'activité
J3 matin	5. Maintenance système et configuration réseau
J3 après-midi	6. Performances et sécurité

13 démos · 7 exercices · 10 TPs sur VM Ubuntu 24.04 ARM64

Architecture système Linux et compilation du noyau

Du userland au kernel, en passant par les modules

Linux - Administration avancée – M2I Formation
Module 1/6

Objectifs du module

À la fin de ce module, vous serez capable de :

- Décrire l'architecture interne d'un système GNU/Linux
- Identifier le rôle des anneaux de protection, du noyau, des bibliothèques et des appels systèmes
- Compiler et installer un noyau Linux personnalisé

Durée : 3h30

Plan

1. Vue d'ensemble de l'architecture
2. Composants : noyau, LKM, rootfs, drivers, libs, syscalls, shells
3. Compiler et installer un noyau Linux
4. Démo et TP
5. Récap et questions

1. Vue d'ensemble de l'architecture

Anneaux de protection x86

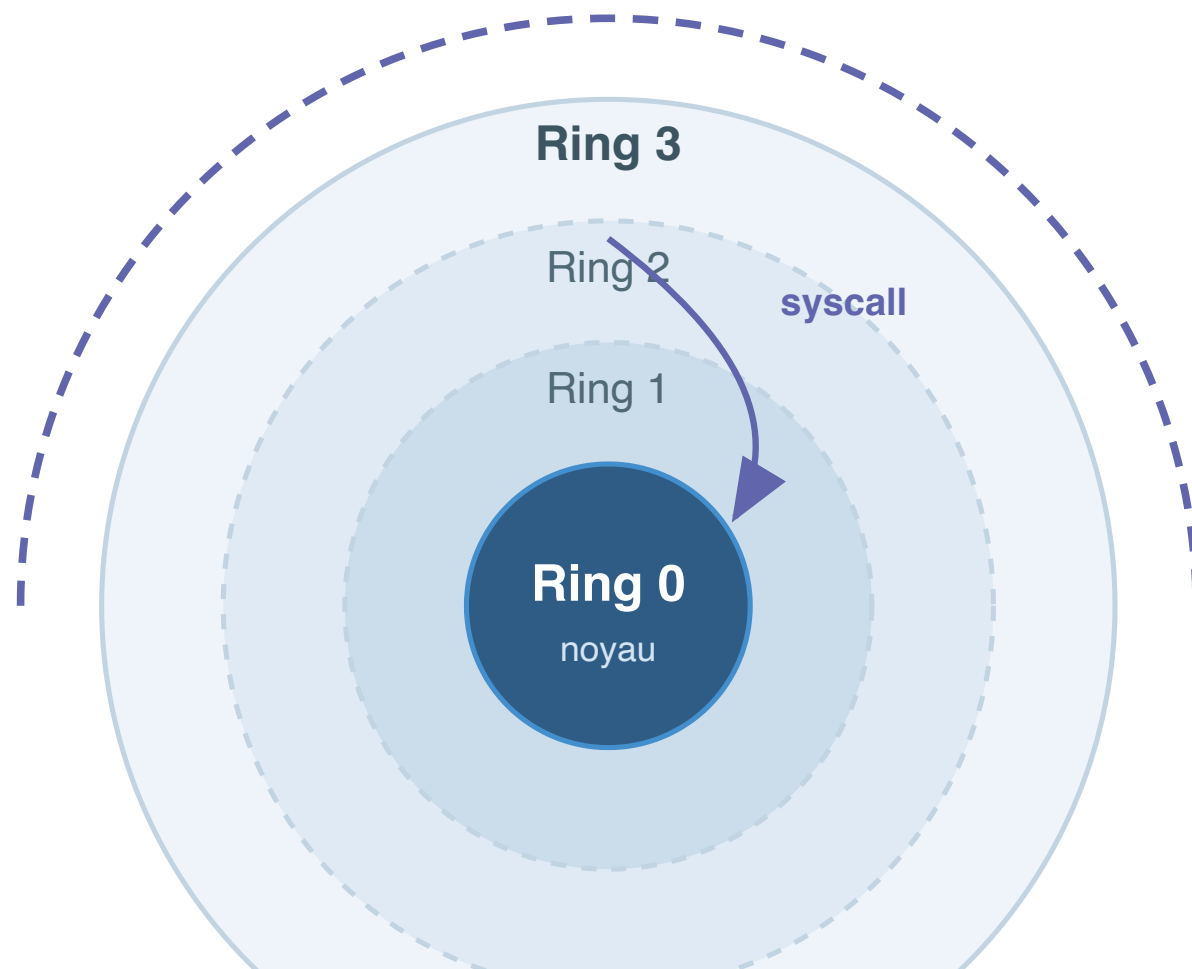
“ Un système d'exploitation moderne sépare strictement ce qui peut s'adresser au matériel de ce qui ne le peut pas. ”

Points essentiels :

- Ring 0 : noyau, accès direct au matériel
- Ring 3 : applications utilisateur (userland)
- Ring -1 : hyperviseur (VT-x, AMD-V) en virtualisation
- Rings 1 et 2 : non utilisés sous Linux

Les anneaux en image

Ring -1 — hyperviseur (VT-x / AMD-V)



■ Ring 0 — kernelland

Accès direct au matériel
Ordonnanceur, mémoire, VFS, drivers

■ Rings 1 et 2

Non utilisés sous Linux

□ Ring 3 — userland

Applications, bibliothèques, shells
Aucun accès direct au matériel

La frontière : les appels systèmes

~400 syscalls sous Linux

Vue d'ensemble GNU/Linux

Userland

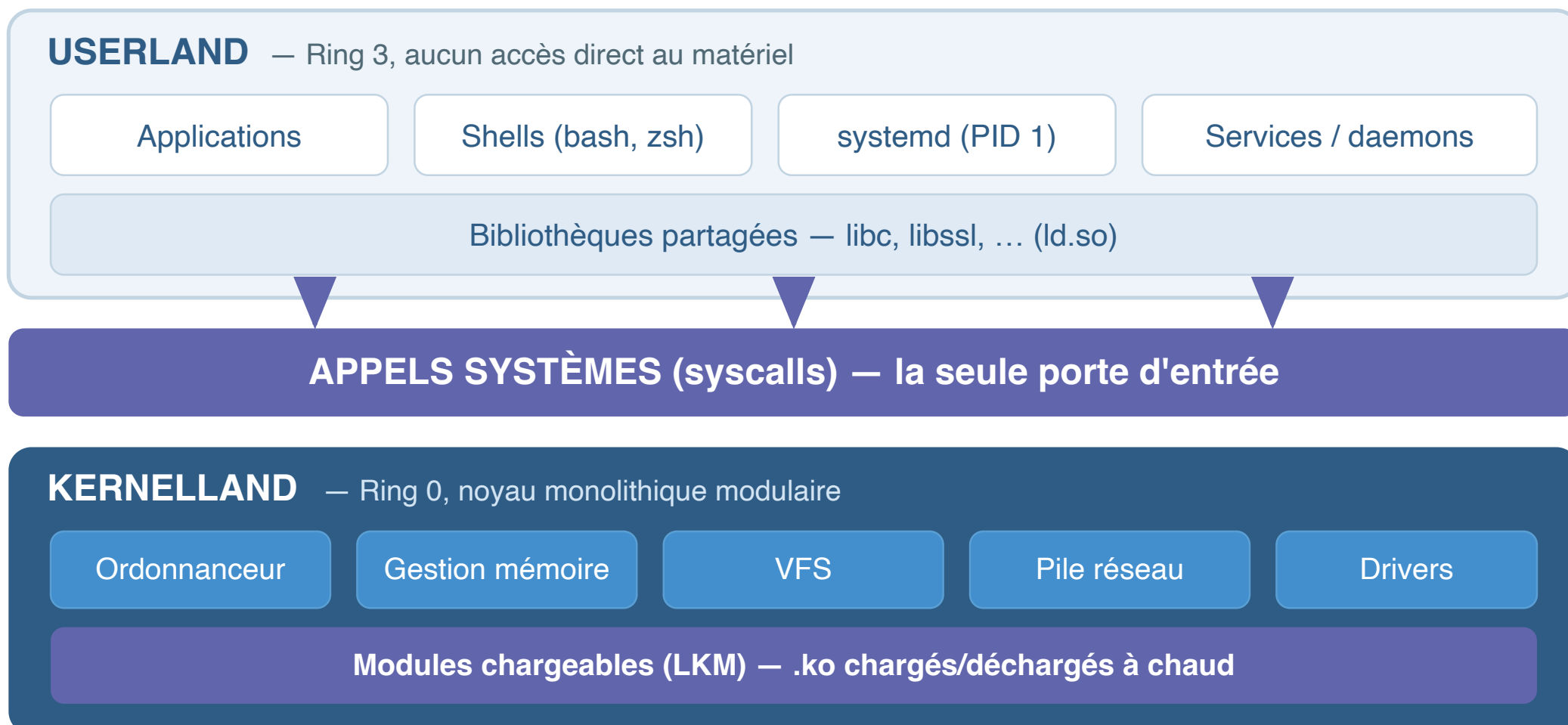
- Applications
- Bibliothèques (libc, libssl)
- Shells (bash, zsh)
- systemd

Kernelland

- Ordonnanceur
- Gestion mémoire
- VFS
- Pilotes (drivers)
- Modules (LKM)

L'interface entre les deux : les **appels systèmes** (`syscall`).

L'empilement complet



Plateformes matérielles supportées

- x86_64 (Intel/AMD) : poste de travail, serveurs
- ARM64 (aarch64) : cloud (Graviton, Ampere), Raspberry Pi 4/5
- RISC-V : émergent
- ppc64le, s390x : mainframes IBM

Le même code source produit un binaire différent par architecture.

```
uname -m  
arch  
file /bin/bash
```

ARM64 vs x86_64 : ce qui change pour l'admin

- Privilèges : les *exception levels* EL0 (userland) / EL1 (noyau) remplacent les rings
- `/proc/cpuinfo` : pas de `model name` ni `flags` — champ `Features`, synthèse via `lscpu`
- Paquets noyau : suffixe `_arm64.deb` (et non `_amd64`)
- Image noyau : `arch/arm64/boot/Image` (pas de `bzImage`)
- Les commandes d'administration restent identiques

Vos VM de TP : **x86_64**. La VM de démo projetée : **ARM64** — d'où

La virtualisation

- **Type 1 (bare metal)** : KVM, ESXi, Xen
- **Type 2 (hosted)** : VirtualBox, VMware Workstation
- **Conteneurs** : namespaces + cgroups + capabilities (LXC, Docker, Podman)

Linux est à la fois hôte (KVM intégré au noyau) et invité standard.

2. Composants du système

Noyau Linux et LKM

Le noyau est **monolithique modulaire** :

- Cœur compilé en un seul binaire (`vmlinux`)
- Fonctions optionnelles chargeables à chaud : **Loadable Kernel Modules** (LKM)
- Drivers, filesystems, protocoles réseau peuvent être en module

```
uname -r           # version active
ls /boot/          # vmlinux-*, initrd*, config-*
lsmod | head       # modules chargés
```

Système de fichiers root (rootfs)

Hiérarchie standard (FHS) :

Chemin	Rôle
/bin, /sbin	binaires essentiels (souvent liés vers /usr/bin)
/etc	configuration
/var	données variables (logs, caches)
/proc	infos noyau (pseudo-FS)
/sys	infos matériel (pseudo-FS)
/dev	nœuds de périphériques

initramfs : démarrer avant le rootfs

Problème de l'œuf et de la poule : pour monter `/`, le noyau a besoin des drivers du disque et du filesystem... souvent stockés en modules **sur `/`**.

Solution : un **mini rootfs en RAM**, chargé par GRUB en même temps que le noyau :

- contient les modules indispensables (stockage, FS, LVM, chiffrement)
- monte le vrai rootfs puis lui passe la main (`switch_root`)

GRUB charge donc toujours deux fichiers : `vmlinux` + `initrd.img`.

initramfs en pratique

```
lsinitramfs /boot/initrd.img-$(uname -r) | head          # contenu  
lsinitramfs /boot/initrd.img-$(uname -r) | grep -c '\.ko'
```

Régénérer après un changement de modules ou de configuration :

```
sudo update-initramfs -u          # noyau courant  
sudo update-initramfs -u -k all   # tous les noyaux installés
```

Blacklister un module sans régénérer l'initramfs = blacklist ignoré au boot.

Pilotes de périphériques

Trois grandes catégories sous Linux :

- **Char devices** : flux d'octets (clavier, série, /dev/random)
 - **Block devices** : par blocs adressables (disques)
 - **Network devices** : ni char ni block, exposés via l'API socket
- Ils peuvent être compilés **dans** le noyau (**y**) ou **en module** (**m**).

Bibliothèques partagées et statiques

- **Statique** (.a) : code embarqué dans le binaire à la compilation
- **Partagée** (.so) : chargée dynamiquement par ld.so à l'exécution

```
ldd /usr/bin/bash      # liste les .so requises
nm -D $(ldd /bin/bash | awk '/libc\.so/{print $3}') | head
ldconfig -p            # cache des libs
```

Avantage du .so : un patch de la libc bénéficie à tous les binaires.

Qui trouve les .so ? Le chargeur ld.so

Au lancement d'un binaire dynamique, `ld.so` résout chaque `.so` dans cet ordre :

1. `DT_RPATH` inscrit dans le binaire (obsolète)
2. Variable d'environnement `LD_LIBRARY_PATH`
3. `DT_RUNPATH` inscrit dans le binaire
4. Cache `/etc/ld.so.cache` (généré par `ldconfig`)
5. Répertoires par défaut : `/lib`, `/usr/lib`

`LD_LIBRARY_PATH` : utile en debug, à bannir comme réglage permanent.

ldconfig et le diagnostic de dépendances

```
ldconfig -p | grep libssl      # la lib est-elle dans le cache ?  
sudo ldconfig                  # régénérer après ajout d'une lib  
cat /etc/ld.so.conf.d/*.conf   # répertoires scannés
```

Symptôme classique au lancement d'une application :

```
error while loading shared libraries: libfoo.so.2:  
cannot open shared object file
```

Réflexe : `ldd ./mon_appli` et repérer les `not found`.

Appels systèmes (syscalls)

Frontière userland / kernel :

```
ssize_t read(int fd, void *buf, size_t count);
```

Tracer les syscalls d'un programme :

```
strace -c ls /tmp  
strace -e openat,read,close cat /etc/hostname
```

Liste complète : `man 2 syscalls` (~400 sous Linux).

Les syscalls que vous croiserez partout

Syscall	Rôle
openat, read, write, close	fichiers et flux
mmap	projection mémoire (libs, fichiers, allocations)
fork / clone	création de processus et de threads
execve	remplacement du programme courant
ioctl	commandes spécifiques à un périphérique

Chaque commande shell est une composition de ces briques.

vDSO : des syscalls sans changer d'anneau

Certains appels très fréquents (`gettimeofday`, `clock_gettime`) ne justifient pas le coût d'une transition vers le ring 0.

- Le noyau projette une mini-bibliothèque dans chaque processus : le **vDSO**
- L'appel se résout en simple lecture mémoire, sans bascule userland → kernel

```
ldd /bin/bash | head -1      # linux-vdso.so.1 : aucun fichier sur disque
grep vdso /proc/self/maps
```

strace : les options qui servent vraiment

```
strace -c ls /tmp                # résumé : compte et durée par syscall
strace -e trace=openat cat /etc/hostname  # filtrer un syscall
strace -e trace=%file ls         # filtrer une famille (fichiers)
strace -f ./script.sh           # suivre aussi les processus fils
strace -p 1234 -o /tmp/trace.log  # s'attacher à un processus vivant
```

`ltrace` fait la même chose au niveau des **appels de bibliothèques** (libc).

Lire une ligne de strace

```
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
read(3, "srv-lab\n", 131072) = 8
close(3) = 0
```

- À gauche : le syscall et ses arguments, décodés par strace
- À droite du `=` : la valeur de retour (fd, octets lus, 0 = succès)
- Une erreur est explicite : `= -1 ENOENT (No such file or directory)`

Shells

Shell	Usage
sh (POSIX)	scripts portables
bash	défaut sur la plupart des distros
zsh	interactif, défaut macOS
dash	rapide, <code>/bin/sh</code> sur Debian/Ubuntu
fish	ergonomie poussée, non POSIX

À retenir : un script `#!/bin/sh` n'est pas équivalent à `#!/bin/bash`.

3. Compiler un noyau Linux

Pourquoi compiler son noyau ?

- Activer un driver absent du noyau distro
- Désactiver des fonctionnalités inutiles (taille, surface d'attaque)
- Appliquer un patch (real-time, sécurité)
- Activer des options de debug
- Comprendre le fonctionnement interne

Cas typique en production : on s'en abstient au profit du noyau distro.

Préparer la compilation

```
sudo apt install build-essential libncurses-dev bison flex \  
    libssl-dev libelf-dev bc kmod cpio  
cd /usr/src  
sudo wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz  
sudo tar xf linux-6.6.tar.xz  
cd linux-6.6
```

Sources officielles : kernel.org.

Configurer

Trois entrées possibles :

```
make defconfig      # config par défaut pour l'archi
make menuconfig     # interface ncurses
make oldconfig      # repart d'un .config existant
```

Le résultat est le fichier `.config` à la racine.

```
cp /boot/config-$(uname -r) .config # repartir du noyau actuel
make olddefconfig                   # accepter les nouveaux défauts
```

menuconfig : y, m ou n ?

Chaque option `CONFIG_*` prend l'une de trois valeurs :

- `y` : compilé **dans** le noyau — toujours présent, disponible dès le boot
- `m` : compilé en **module** — chargé à la demande
- `n` : absent

```
grep CONFIG_EXT4 .config      # =y : indispensable pour monter le rootfs
make localmodconfig           # ne garde que les modules charges
```

`localmodconfig` réduit fortement la durée de compilation d'un noyau d'étude.

Compiler et installer (méthode classique)

```
make -j$(nproc)           # compilation
sudo make modules_install  # /lib/modules/<version>/
sudo make install          # /boot/vmlinuz-*, update-grub
```

À l'issue :

```
ls /boot/
sudo update-grub
sudo reboot
```

Méthode Debian (paquets .deb)

Plus propre pour la maintenance :

```
# kernel-package a ete retire de Debian/Ubuntu : on utilise la cible  
# bindeb-pkg fournie directement par les sources du noyau.  
make -j$(nproc) bindeb-pkg  
ls ../linux-image-*.deb  
sudo dpkg -i ../linux-image-*.deb ../linux-headers-*.deb
```

Avantage : `apt purge` permet de retirer le noyau proprement.

La chaîne complète en une slide

```
.config → make -j$(nproc) bindeb-pkg → ../linux-image-*_amd64.deb  
                                         |  
uname -r ← reboot ← update-grub ← dpkg -i (+ initramfs)
```

- Le `.config` décide de tout : `y` / `m` / `n`
- `bindeb-pkg` produit des paquets `_<arch>.deb` désinstallables (`_amd64` sur vos VM, `_arm64` sur la démo)
- `dpkg -i` déclenche la génération de l'initramfs et `update-grub`

Intégration de drivers et outils tiers

- **DKMS** (Dynamic Kernel Module Support) : recompile auto à chaque maj
- **out-of-tree** : driver vendor (NVIDIA, ZFS) non inclus en upstream
- **firmware** : `linux-firmware` (binaires propriétaires des cartes WiFi, GPU)

```
dkms status  
ls /lib/modules/$(uname -r)/extra/ 2>/dev/null # absent par défaut
```

DKMS : le geste industriel

Problème : un module out-of-tree compilé pour `6.8.0-40` **casse** à la mise à jour du noyau vers `6.8.0-41`.

DKMS (Dynamic Kernel Module Support) :

- conserve les **sources** du module dans `/usr/src/<nom>-<version>/`
- recompile automatiquement à chaque installation de noyau (hook APT)
- c'est le mécanisme des drivers NVIDIA, ZFS, VirtualBox...

DKMS : le cycle en commandes

```
sudo apt install dkms
# sources dans /usr/src/hello-1.0/ avec un fichier dkms.conf
sudo dkms add hello/1.0          # enregistre le module
sudo dkms build hello/1.0        # compile pour le noyau actif
sudo dkms install hello/1.0      # installe dans /lib/modules/...
dkms status                      # etat par noyau installe
```

À chaque mise à jour du noyau, DKMS rejoue `build` + `install` tout seul.

Démo live

Démo 1 : exploration de l'architecture

Démo 2 : compilation d'un noyau Linux

Voir `demos/demo-01.md` et `demos/demo-02.md`

Travaux à suivre

Exercice 1 : Cartographier les composants d'un système Linux

- Durée : 30 min — Fichier : `exercices/exercice-01.md`

TP 1 : Compiler et installer un noyau personnalisé en méthode Debian

- Durée : 1h30 — Fichier : `tp/tp-01.md`

Récap du module

- Linux sépare strictement userland et kernel via les syscalls
- Le noyau est monolithique modulaire : ce qui n'est pas embarqué peut être un LKM
- Compiler un noyau = `make defconfig`, `make`, `make modules_install`,
`make install`
- En production : préférer un noyau distro packagé

Cheatsheet : `cheatsheet/module-01.md`

Questions

Module suivant : Modules de noyau, /proc, /sys et dépannage matériel

Modules de noyau, /proc, /sys et dépannage matériel

Étendre, observer et diagnostiquer Linux

Linux - Administration avancée — M2I Formation

Module 2/6

Objectifs du module

À la fin de ce module, vous serez capable de :

- Concevoir, compiler et gérer des modules de noyau dynamiques
- Exploiter les pseudo-systèmes /proc et /sys pour configurer et diagnostiquer
- Diagnostiquer un problème matériel et identifier les incidents associés

Durée : 3h30

Plan

1. Loadable Kernel Modules (LKM)
2. Pseudo-systèmes /proc et /sys
3. Dépannage matériel
4. Démo et TP
5. Récap et questions

1. Loadable Kernel Modules

Pourquoi un module et pas du built-in ?

- Réduire la taille du noyau chargé en mémoire
- Charger uniquement ce qui est utilisé
- Permettre la maintenance d'un driver sans rebooter
- Distribuer un driver tiers (NVIDIA, ZFS, drivers vendor)

Un module peut être chargé/déchargé à chaud (sauf dépendance bloquante).

Anatomie d'un module minimal

```
#include <linux/module.h>
#include <linux/kernel.h>

static int __init hello_init(void) {
    pr_info("Hello from LKM\n");
    return 0;
}

static void __exit hello_exit(void) {
    pr_info("Goodbye from LKM\n");
}

module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
```

Compiler un module out-of-tree

Makefile minimal :

```
obj-m += hello.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Compilation :

```
make
ls -l hello.ko
modinfo hello.ko
```

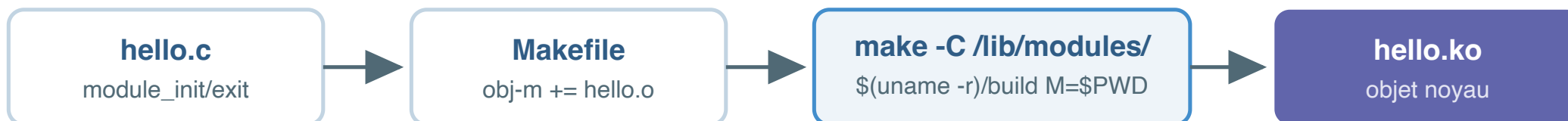
Charger, décharger, inspecter

```
sudo insmod ./hello.ko      # charge un .ko précis
sudo modprobe nfs           # charge avec ses dépendances
sudo rmmod hello            # décharge
lsmod | grep hello          # vérifie
modinfo hello               # métadonnées
dmesg | tail -5             # messages noyau
```

```
modprobe consulte /lib/modules/$(uname -r)/modules.dep.
```

Du code source au module chargé

1 — CONSTRUCTION (userland)



Le .ko est compilé contre les headers du noyau ACTIF : changer de noyau impose de recompiler (d'où DKMS).

2 — CYCLE DE VIE (kernelland, Ring 0)



Charger : deux voies

insmod ./hello.ko

chemin explicite, AUCUNE dépendance résolue

Inspecter et bloquer

lsmod · modinfo · dmesg

état, métadonnées, messages printk du module

Installer un module dans le système

Un `insmod ./hello.ko` ne survit pas au reboot. Pour l'intégrer proprement :

```
sudo mkdir -p /lib/modules/$(uname -r)/extra # absent par défaut
sudo cp hello.ko /lib/modules/$(uname -r)/extra/
sudo depmod -a # met à jour modules.dep
sudo modprobe hello # charge par son nom
```

Sans `depmod -a`, `modprobe` ignore l'existence du nouveau module.

Gestion des dépendances

```
sudo depmod -a          # régénère modules.dep  
modprobe --show-depends nfs # arbre de dépendances
```

Bloquer un module :

```
echo "blacklist nouveau" | sudo tee /etc/modprobe.d/blacklist-nouveau.conf  
sudo update-initramfs -u
```

Charger automatiquement au boot :

```
echo "br_netfilter" | sudo tee /etc/modules-load.d/br_netfilter.conf
```

Paramètres de modules

Lister les paramètres et leurs valeurs courantes :

```
modinfo -p e1000  
ls /sys/module/e1000/parameters/  
ls /sys/module/virtio_net/parameters/    # e1000 n'existe pas en VM virtio
```

Définir au chargement :

```
sudo modprobe e1000 debug=4  
# ou de manière persistante  
echo "options e1000 debug=4" | sudo tee /etc/modprobe.d/e1000.conf
```

Le taint : un noyau marqué

Charger certains modules **marque** durablement le noyau comme "tainted" :

- module propriétaire (P), chargement forcé (F)
- module out-of-tree (O), module non signé (E)
- erreur matérielle sérieuse (MCE) ou oops déjà survenu

Conséquences :

- les rapports d'oops deviennent inexploitable pour les développeurs upstream
- le **support éditeur** (Canonical, Red Hat, SUSE) peut être refusé

Lire et interpréter le taint

```
cat /proc/sys/kernel/tainted    # 0 = noyau propre
```

La valeur est un champ de bits. Les plus courants :

Bit	Valeur	Lettre	Signification
0	1	P	module propriétaire
12	4096	O	module out-of-tree
13	8192	E	module non signé

Après chargement de votre `hello.ko` : `12288` (O + E) — c'est attendu.

Secure Boot et modules signés

Avec **Secure Boot** (UEFI) actif, la chaîne de confiance descend jusqu'aux modules :

- le firmware ne charge qu'un bootloader signé (shim)
- qui ne charge qu'un noyau signé
- qui n'accepte que des **modules signés** par une clé de confiance

Un `.ko` maison non signé échoue alors au chargement :

```
insmod: ERROR: could not insert module hello.ko:  
Key was rejected by service
```

MOK : enregistrer sa propre clé

MOK (Machine Owner Key) : ajouter sa clé à la liste de confiance UEFI.

```
mokutil --sb-state          # Secure Boot actif ?  
sudo mokutil --import MOK.der # enregistrement confirme au prochain boot  
/usr/src/linux-headers-$(uname -r)/scripts/sign-file \  
    sha256 MOK.priv MOK.der hello.ko
```

DKMS sait signer automatiquement les modules avec une clé MOK.
La VM du lab n'a **pas** de Secure Boot : `insmod` passe sans signature.

2. /proc et /sys

/proc — vue noyau et processus

Pseudo-système de fichiers, contenu généré dynamiquement :

Chemin	Contenu
/proc/cpuinfo	CPU, modèle, flags
/proc/meminfo	mémoire, swap, slabs
/proc/mounts	montages effectifs
/proc/cmdline	ligne de commande du noyau
/proc/<pid>/	tout sur un processus
/proc/sys/	paramètres ajustables (sysctl)

/proc/PID : l'identité d'un processus

```
head -6 /proc/$$/status          # etat, PID, memoire (VmRSS)
tr '\0' ' ' < /proc/$$/cmdline; echo
tr '\0' '\n' < /proc/$$/environ | head -3
ls -l /proc/$$/cwd /proc/$$/exe   # repertoire courant, binaire reel
```

- `cmdline` et `environ` sont séparés par des octets nuls, d'où `tr`
- `exe` pointe vers le binaire, même supprimé du disque entre-temps

/proc/PID : fd, maps, limits

```
ls -l /proc/$$/fd          # descripteurs ouverts (fichiers, sockets)
head -5 /proc/$$/maps      # segments memoire : binaire, libs, [vdso]
head -5 /proc/$$/limits    # ulimits effectifs du processus
```

Cas d'usage : retrouver **quel fichier** un processus bloqué garde ouvert, ou vérifier la limite `open files` réellement appliquée à un démon.

Lire /proc/loadavg

```
cat /proc/loadavg  
0.52 0.34 0.19 2/407 12345
```

- Trois moyennes de charge : 1, 5 et 15 minutes
- Charge = processus **exécutables** + en attente d'I/O non interruptible
- **2/407** : threads en cours d'exécution / total ; **12345** : dernier PID créé
- Repère : charge durablement supérieure au nombre de cœurs = saturation

/proc/meminfo : MemFree n'est pas "libre"

```
grep -E 'MemTotal|MemFree|MemAvailable|^Cached' /proc/meminfo
```

- **MemFree** : mémoire strictement inutilisée — **toujours faible**, c'est normal
- **Cached** : cache disque, récupérable instantanément
- **MemAvailable** : estimation de ce qui est réellement mobilisable

Règle : surveiller **MemAvailable**, jamais **MemFree**.

free -h colonne **available** lit exactement cette valeur.

sysctl – ajuster le noyau au runtime

Lire :

```
sysctl -a | grep ip_forward  
cat /proc/sys/net/ipv4/ip_forward
```

Écrire ponctuellement :

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Persister :

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/90-routing.conf  
sudo sysctl --system
```

sysctl : l'ordre de résolution

Fichiers pris en compte par `sysctl --system` (et au boot) :

1. `/usr/lib/sysctl.d/*.conf` — valeurs du distributeur
2. `/run/sysctl.d/*.conf` — réglages runtime
3. `/etc/sysctl.d/*.conf` — **vos réglages**
4. `/etc/sysctl.conf` — emplacement historique

Tri par **nom**, dernière valeur lue gagne ; même nom dans `/etc` masque `/usr/lib`.

Tri lexical : `9-a.conf` passe **avant** `90-a.conf` — préfixez à deux

/sys – vue matériel et drivers

sysfs expose les **objets noyau** (devices, drivers, classes) :

Chemin	Contenu
/sys/class/net/	interfaces réseau
/sys/block/	disques, partitions
/sys/devices/	hiérarchie bus → device
/sys/module/	modules chargés et paramètres
/sys/power/	suspend, hibernate

Souvent en lecture/écriture pour piloter l'état d'un périphérique.

sysfs en pratique

```
# état d'une interface (adapter le nom : ip -br link)
IF=$(ip -br link | awk '$1!="lo"{print $1; exit}')
cat /sys/class/net/$IF/operstate
cat /sys/class/net/$IF/mtu

# governor CPU – absent en VM et sur la plupart des ARM64
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor 2>/dev/null \
    || echo "pas de cpufreq (machine virtualisée)"

# luminosité écran – laptop physique uniquement
ls /sys/class/backlight/
```

En VM, `cpufreq` et `backlight` sont vides : le matériel est emulé.

sysfs : lire un lien de bout en bout

```
IF=$(ip -br link | awk '$1!="lo"{print $1; exit}')
```

du nom logique vers le peripherique physique

```
readlink -f /sys/class/net/$IF
```

quel driver pilote cette interface ?

```
basename $(readlink -f /sys/class/net/$IF/device/driver)
```

C'est le chemin que suit `udev` pour nommer vos interfaces.
`systemd` (paquet `systemd`) offre une vue conviviale.

udev : donner vie à sysfs

Quand le noyau détecte un périphérique, il crée l'objet dans `/sys` et émet un **uevent**. Le démon `udev` le reçoit et :

- crée les nœuds dans `/dev` (noms, permissions, liens symboliques)
- applique les **règles** : `/usr/lib/udev/rules.d/` puis `/etc/udev/rules.d/`
- peut charger le module adapté (via l'alias `modalias`)

C'est pourquoi brancher un disque USB fait apparaître `/dev/sdb` tout seul.

udevadm : inspecter et suivre

```
IF=$(ip -br link | awk '$1!="lo"{print $1; exit}')
```

```
udevadm info /sys/class/net/$IF | head -8    # proprietes du device
```

```
sudo udevadm monitor                          # evenements en direct
```

Exemple de règle (`/etc/udev/rules.d/70-lan.rules`) :

```
SUBSYSTEM=="net", ATTR{address}=="52:54:00:aa:bb:cc", NAME="lan0"
```

Recharger : `sudo udevadm control --reload`.

3. Dépannage matériel

Types de problèmes matériels

- Disque qui pré-tombe en panne (SMART)
- Mémoire défectueuse (erreurs ECC, segfaults erratiques)
- Overheat CPU et throttling
- Driver incorrect ou non chargé
- Câble/port réseau, bus PCIe défaillant
- Firmware obsolète

Première étape : isoler matériel vs logiciel via les journaux.

Inventaire matériel

```
lscpu          # CPU, cores, flags, virtualisation
lsmem          # blocs mémoire
free -h        # mémoire totale et libre
lsblk -f       # disques et partitions
lspci -nnk     # bus PCI + pilote utilisé
lsusb          # périphériques USB
dmidecode -t system # infos BIOS/DMI (root)
```

Surveillance santé matérielle

```
# disque
sudo smartctl -a /dev/sda
sudo smartctl -t short /dev/sda

# mémoire
sudo memtester 1G 1
# ou memtest86+ au boot

# températures et capteurs
sensors
sudo sensors-detect
```

Journaux noyau

```
dmesg -T | tail -50  
journalctl -k --since "10 min ago"  
journalctl -p err -k -b
```

Mots-clés à repérer : I/O error, Hardware Error, MCE, Out of memory, link is down.

dmesg : l'arbre de décision

Mot-clé dans dmesg	Suspicion	Outil suivant
I/O error, ata error	disque	smartctl -a
MCE, Hardware Error	CPU / RAM	rasdaemon, memtest
thermal, throttled	refroidissement	sensors
Out of memory	logiciel, dimensionnement	journalctl, cgroups
link is down	câble / switch / carte	ethtool

Le mot-clé oriente ; l'outil spécialisé confirme.

En VM : le matériel est une illusion

En VM, quelle que soit l'architecture, le matériel est émulé. Sur la VM de démo (ARM64, **virtio**) :

- `smartctl /dev/vda` échoue : pas de SMART sur un disque virtuel
- `sensors` ne trouve rien : aucun capteur exposé à l'invité
- `dmidecode` renvoie des données d'émulation

À retenir : en VM, le diagnostic *matériel* relève de l'**hyperviseur** et de l'hôte. L'invité ne voit que l'étage logiciel — exactement la frontière vue au module 1.

Démo live

Démo 3 : compilation et chargement d'un LKM minimal

Démo 4 : tour des inspections /proc, /sys et sysctl

Voir `demos/demo-03.md` et `demos/demo-04.md`

Travaux à suivre

Exercices

- Exercice 2 : Interpréter `/proc` et `/sys` (`exercices/exercice-02.md`) — 30 min
- Exercice 3 : Paramètres noyau persistants via `sysctl` (`exercices/exercice-03.md`) — 25 min

TPs

- TP 2 : Compiler et installer un module de noyau (`tp/tp-02.md`) — 1h
- TP 3 : Audit matériel et identification d'incidents (`tp/tp-03.md`) — 45 min

Récap du module

- Un LKM s'écrit en C, se compile contre les headers, se charge avec `modprobe`
- `/proc` expose l'état noyau et processus ; `sysctl` permet d'ajuster les paramètres
- `/sys` reflète la hiérarchie des bus et drivers et permet de piloter le matériel
- Le diagnostic matériel commence toujours par `dmesg` et les outils d'inventaire

Cheatsheet : `cheatsheet/module-02.md`

Questions

Module suivant : Stockage avancé — LVM et BTRFS

Stockage avancé : LVM et BTRFS

Volumes logiques, snapshots et systèmes de fichiers modernes

Linux - Administration avancée — M2I Formation
Module 3/6

Objectifs du module

À la fin de ce module, vous serez capable de :

- Comparer les principaux systèmes de fichiers Linux
- Maîtriser LVM (PV, VG, LV, PE/LE, métadonnées, sécurisation)
- Mettre en oeuvre BTRFS (subvolumes, snapshots, CoW, compression)

Durée : 3h30

Plan

1. Systèmes de fichiers Linux
2. LVM et Device Mapper
3. BTRFS
4. Démo et TP
5. Récap et questions

1. Systèmes de fichiers

Panorama des FS

FS	Forces	Limites
ext2	simplicité, peu de meta	pas de journal
ext3	journal, compat ext2	perfs limitées
ext4	extents, large FS, défaut Debian	pas de CoW
XFS	très grands FS, parallélisme	shrink impossible
ZFS	CoW, RAID, snapshots, intégrité	hors arbre noyau (CDDL)
BTRFS	CoW, snapshots, subvolumes	RAID5/6 fragile

Choix par cas d'usage, pas universel.

Critères de choix

- **Taille** : XFS à l'aise sur les très gros volumes ; ext4 monte à 1 EiB (le « 16 To » n'est que le plafond par défaut en blocs 4K)
- **Charge** : XFS pour I/O parallèles, ext4 pour mixte
- **Snapshots/CoW** : BTRFS ou ZFS
- **Stabilité éprouvée** : ext4
- **Compression intégrée** : BTRFS, ZFS

```
mkfs.ext4 -L data /dev/sdb1  
mkfs.xfs -L bigdata /dev/sdb1
```

fstab : identifier les volumes

Méthode	Exemple	Robustesse
Chemin device	/dev/sdb1	fragile : l'ordre de détection change
UUID	UUID=3c5a-...	stable, généré au mkfs
LABEL	LABEL=data	stable et lisible
Chemin LVM	/dev/vg_data/lv_app	stable (Device Mapper)

```
sudo blkid /dev/sdb1
lsblk -f
```

fstab : options de montage

```
LABEL=data /mnt/data btrfs compress=zstd:3,noatime 0 0
```

- `defaults` : rw, suid, dev, exec, auto, nouser, async
- `noatime` : pas de mise à jour des dates d'accès (perfs)
- `nofail` : le boot continue si le volume est absent
- `x-systemd.automount` : montage au premier accès seulement
- Champs 5 et 6 : dump et ordre de `fsck` (0 = jamais)

Toujours valider avec `sudo mount -a` **avant** de rebooter.

Quel FS choisir ?

Besoin de snapshots / compression intégrés ?

└─ oui → licence CDDL acceptable, RAM abondante ?

└─ oui → ZFS

└─ non → BTRFS

└─ non → très gros volume, I/O massivement parallèles ?

└─ oui → XFS

└─ non → ext4 (défaut sûr)

En cas de doute : ext4. Un choix de FS se vit pendant des années.

2. LVM et Device Mapper

Pourquoi LVM ?

Sans LVM : 1 partition = 1 FS de taille fixe.

Avec LVM :

- Agréger plusieurs disques en un pool (VG)
- Découper en LV redimensionnables à chaud
- Snapshots, mirroring, thin provisioning
- Migration de données entre PV sans démontage

LVM s'appuie sur **Device Mapper** dans le noyau.

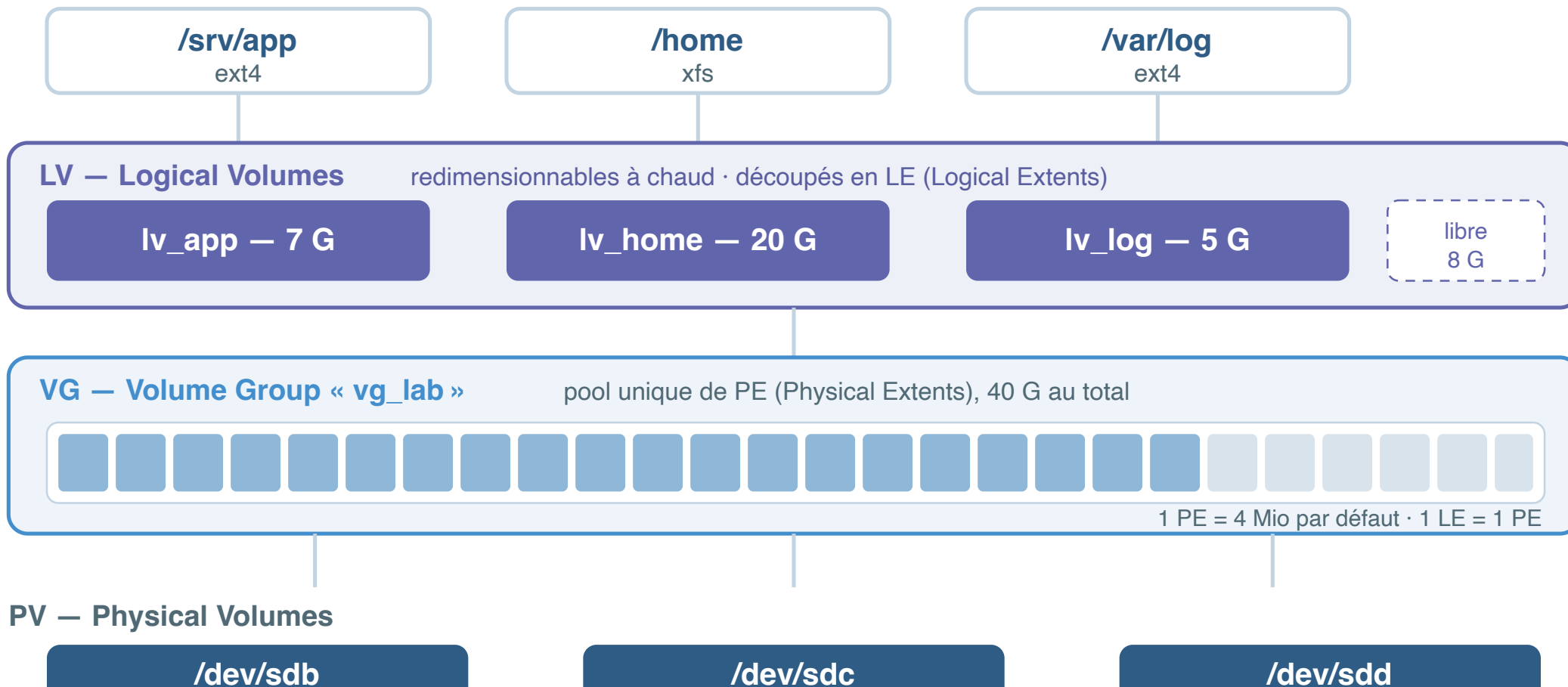
Vocabulaire LVM

```
+-----+
|               Volume Group (VG)               |
|                                                 |
|  PV (Physical Volume) : un disque/partition    |
|  PE (Physical Extent) : unité d'allocation      |
|                                                 |
|  LV (Logical Volume) : volume utilisable        |
|  LE (Logical Extent) : 1 LE = 1 PE              |
+-----+
```

Métadonnées LVM2 : un *label header* (secteur 1) puis une ou deux *metadata areas* (MDA) en début de PV — sauvegardées dans `/etc/lvm/backup/`.

L'empilement LVM

POINTS DE MONTAGE



Créer un VG et un LV

```
sudo pvcreate /dev/sdb /dev/sdc
sudo vgcreate vg_data /dev/sdb /dev/sdc

sudo lvcreate -n lv_app -L 20G vg_data
sudo mkfs.ext4 /dev/vg_data/lv_app

sudo mkdir /srv/app
echo "/dev/vg_data/lv_app /srv/app ext4 defaults 0 2" | sudo tee -a /etc/fstab
sudo mount -a
```

Inspector LVM

```
sudo pvs          # vue Physical Volumes
sudo vgs          # vue Volume Groups
sudo lvs          # vue Logical Volumes
```

```
sudo pvdisplay
sudo vgdisplay vg_data
sudo lvdisplay /dev/vg_data/lv_app
```

Format colonnes (script) :

```
sudo lvs -o lv_name,lv_size,lv_attr,vg_name --noheadings
```

Étendre un volume logique

```
# 1. ajouter un PV au besoin
sudo pvcreate /dev/sdd
sudo vgextend vg_data /dev/sdd

# 2. étendre le LV
sudo lvextend -L +10G /dev/vg_data/lv_app

# 3. étendre le FS (ext4)
sudo resize2fs /dev/vg_data/lv_app
# ou XFS : sudo xfs_growfs /srv/app
```

Tout cela à chaud, sans démonter.

`lvextend -r` enchaîne les étapes 2 et 3 en une commande.

Mirroring LVM : survivre à un disque

```
# LV en deux copies, sur deux PV distincts
sudo lvcreate --type raid1 -m 1 -n lv_sec -L 5G vg_data

# convertir un LV existant en miroir
sudo lvconvert --type raid1 -m 1 vg_data/lv_app

sudo lvs -a -o name,copy_percent,devices vg_data
```

Perte d'un PV : le LV reste accessible en mode dégradé.

Réparation : `lvconvert --repair` avec un PV de remplacement.

pvmove : migrer un PV à chaud

Retirer un disque vieillissant sans interruption de service :

```
sudo pvmove /dev/sdb          # évacue tous les PE du PV
sudo vgreduce vg_data /dev/sdb # sortir le PV du VG
sudo pvremove /dev/sdb        # effacer les métadonnées LVM
```

- Fonctionne LV montés, applications en production
- Interrompu ? `pvmove` sans argument reprend où il en était
- `pvmove /dev/sdb /dev/sdd` cible le PV de destination

Snapshots LVM

```
sudo lvcreate -L 5G -s -n snap_app /dev/vg_data/lv_app

# faire la sauvegarde depuis le snapshot
sudo mount -o ro /dev/vg_data/snap_app /mnt/snap
tar czf backup.tgz -C /mnt/snap .
sudo umount /mnt/snap
sudo lvremove /dev/vg_data/snap_app
```

Snapshot LVM = CoW au niveau bloc, taille à dimensionner pour les écritures durant la vie du snap.

Saturation d'un snapshot LVM

La zone CoW d'un snapshot est de **taille fixe**, figée à la création.

- Chaque bloc modifié sur l'origine y est copié
- Zone pleine → snapshot **invalide**, attribut **I** dans **lvs**
- Un snapshot invalide est irrécupérable : la sauvegarde échoue

```
sudo lvs -o name,attr,data_percent vg_data
# snap_app swi-I-s--- 100.00 <- trop tard
```

Dimensionner ~20 % de l'origine, surveiller **Data%** pendant la vie du snap.

Thin provisioning : le principe

Promettre **plus que ce qu'on possède**, ne consommer qu'à l'écriture.

- Un **thin pool** : réserve réelle de blocs prise dans le VG
- Des **LV thin** : tailles virtuelles, leur somme peut dépasser le pool
- Les blocs ne sont prélevés qu'au moment des écritures

Cas d'usage : parcs de VM, environnements de test, snapshots nombreux.

Contrepartie : un pool plein = erreurs d'I/O sur **tous** les LV thin.

Créer un thin pool et des LV thin

```
# le pool : 8 Go réels pris dans le VG
sudo lvcreate --type thin-pool -L 8G -n pool0 vg_data

# deux LV virtuels de 10 Go (20 Go promis, 8 Go réels)
sudo lvcreate --type thin -V 10G --thinpool pool0 -n lv_vm1 vg_data
sudo lvcreate --type thin -V 10G --thinpool pool0 -n lv_vm2 vg_data

sudo lvs vg_data
```

La colonne **Data%** du pool est la seule vérité sur l'espace restant.

Surveiller un pool thin

```
sudo lvs -o name,lv_size,data_percent,metadata_percent vg_data
```

- Alerte dès **80 %** d'occupation (data **ou** metadata)
- Auto-extension possible via `/etc/lvm/lvm.conf` :

```
thin_pool_autoextend_threshold = 80  
thin_pool_autoextend_percent = 20
```

Piège : `df -h` dans les LV ne voit rien venir — la saturation se joue au niveau bloc, pas au niveau FS.

Chiffrer avec LUKS

Chiffrement au niveau bloc, sous le FS. Deux placements :

- **LUKS sur le PV** : tout le VG est chiffré, une seule passphrase
- **LUKS sur un LV** : chiffrement ciblé, volume par volume

```
sudo cryptsetup luksFormat /dev/vg_data/lv_secret  
sudo cryptsetup open /dev/vg_data/lv_secret secret  
sudo mkfs.ext4 /dev/mapper/secret
```

open crée un device Device Mapper en clair : /dev/mapper/secret.

LUKS au démarrage : crypttab

`/etc/crypttab` déclare les volumes à ouvrir au boot :

```
# nom      device                  clé    options
secret    /dev/vg_data/lv_secret  none   luks
```

- `none` : passphrase demandée à la console au démarrage
- Fichier de clé possible pour les volumes secondaires
- Le fstab référence ensuite `/dev/mapper/secret`

Sauvegarder l'en-tête : `cryptsetup luksHeaderBackup` (sans lui, les données sont perdues).

Sécuriser LVM

- **Mirroring** : `lvcreate --type raid1 -m 1` (2 copies)
- **Thin provisioning** : surveiller le pool, alertes à 80%
- **Métadonnées** : `vgcfgbackup` quotidien
- **Chiffrement** : LUKS sur le PV ou sur le LV

```
sudo vgcfgbackup vg_data          # /etc/lvm/backup/  
sudo vgcfgrestore -f ... vg_data # en récupération
```

3. BTRFS

Pourquoi BTRFS ?

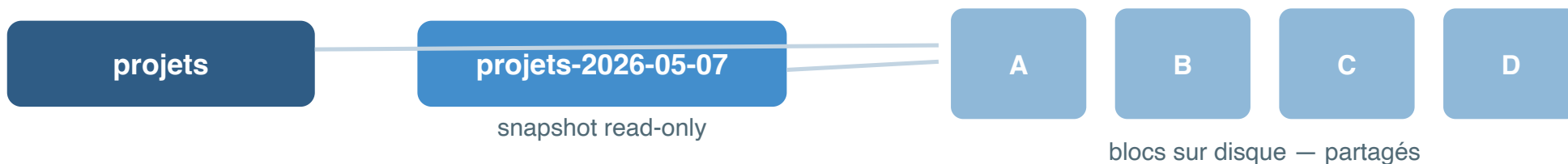
- **CoW** (Copy-on-Write) natif
- Snapshots quasi instantanés et peu coûteux
- Subvolumes (FS dans le FS)
- Compression transparente (zstd, lzo)
- Checksums sur data et metadata
- Conversion ext4 → BTRFS sans format

Limites : RAID5/6 instable, performances mixtes sur DB.

Copy-on-Write : ce que coûte un snapshot

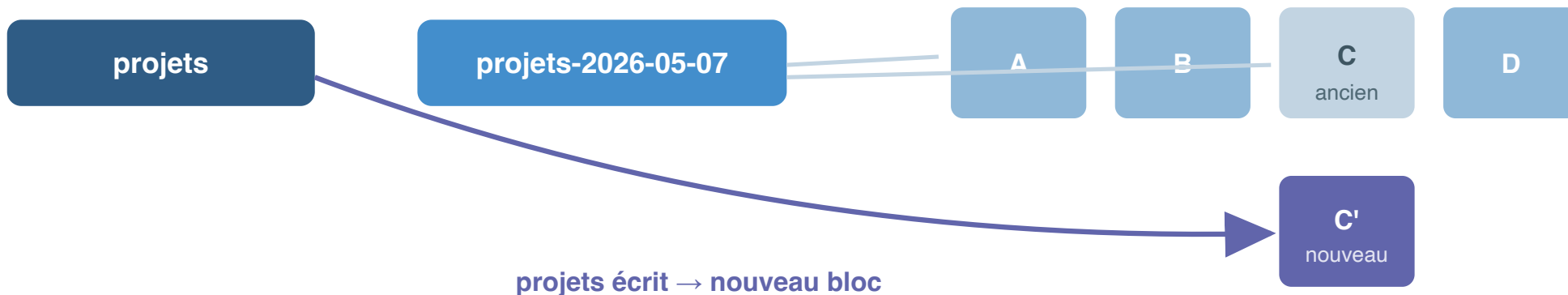
1 — Snapshot créé : rien n'est copié

Le snapshot pointe vers les mêmes blocs. Coût disque ≈ 0 , création quasi instantanée.



2 — Écriture dans « projets » : Copy-on-Write

Le bloc C n'est jamais écrasé. BTRFS écrit un nouveau bloc C' et réoriente le pointeur de « projets ».



Créer un FS BTRFS

```
sudo mkfs.btrfs -L data /dev/sdb1
sudo mkdir /mnt/btrfs
sudo mount -o compress=zstd /dev/sdb1 /mnt/btrfs

# subvolumes
sudo btrfs subvolume create /mnt/btrfs/projets
sudo btrfs subvolume create /mnt/btrfs/backups
sudo btrfs subvolume list /mnt/btrfs
```

Compression : mesurer juste

`du -sh` ne voit **pas** la compression : il compte des tailles logiques et affiche parfois plus gros que la source.

Les deux seuls indicateurs fiables :

```
sudo compsize /mnt/btrfs/projets      # paquet btrfs-compsize  
sudo btrfs filesystem df /mnt/btrfs   # ligne "Data ... used"
```

`compsize` donne le ratio réel, algorithme par algorithme (zstd, lzo, none).

Snapshots BTRFS

```
# snapshot read-only
sudo btrfs subvolume snapshot -r \
    /mnt/btrfs/projets /mnt/btrfs/projets-2026-05-07

# rollback : remonter le snapshot
sudo mount -o subvol=projets-2026-05-07 /dev/sdb1 /mnt/projets-restore

# supprimer
sudo btrfs subvolume delete /mnt/btrfs/projets-2026-05-07
```

Coût quasi nul à la création, croît avec les écritures post-snap.

Un snapshot n'est pas une sauvegarde

Le snapshot vit **dans le même FS**, sur le même device : si le disque meurt, l'original et tous ses snapshots meurent ensemble.

`btrfs send/receive` exporte un snapshot vers un **autre** FS BTRFS :

```
sudo btrfs subvolume snapshot -r /mnt/btrfs/projets /mnt/btrfs/projets-ref  
sudo btrfs send /mnt/btrfs/projets-ref | sudo btrfs receive /mnt/backup/
```

Prérequis : le snapshot source est **read-only** (`-r`).

send/receive incrémental

Après une première copie complète, n'envoyer que le delta :

```
sudo btrfs subvolume snapshot -r /mnt/btrfs/projets /mnt/btrfs/projets-j2  
  
sudo btrfs send -p /mnt/btrfs/projets-ref /mnt/btrfs/projets-j2 | \  
sudo btrfs receive /mnt/backup/
```

- `-p` : snapshot parent, présent des deux côtés
- Le flux peut traverser `ssh` vers un hôte distant
- C'est la mécanique des outils comme `btrbk` ou `snapper` + scripts

Inspector BTRFS

```
sudo btrfs filesystem show
sudo btrfs filesystem df /mnt/btrfs
sudo btrfs filesystem usage /mnt/btrfs

sudo btrfs scrub start /mnt/btrfs
sudo btrfs scrub status /mnt/btrfs
```

scrub lit l'ensemble des données et vérifie les checksums.

Snapshot LVM vs snapshot BTRFS

Critère	LVM	BTRFS
Niveau	bloc (Device Mapper)	FS (CoW natif)
Coût à la création	zone CoW à réserver	quasi nul
Croissance	zone fixe , se remplit	libre, dans le FS
Saturation	snapshot invalide (I)	remplit le FS entier
Rollback	<code>lvconvert --merge</code>	monter / renommer le subvolume
FS supportés	tous	BTRFS uniquement

Démo live

Démo 5 : créer un VG/LV monté de façon persistante

Démo 6 : BTRFS avec subvolume, snapshot et compression

Voir `demos/demo-05.md` et `demos/demo-06.md`

Travaux à suivre

Exercice 4 : Interpréter la sortie de `pvs` / `vgs` / `lvs`

- Durée : 30 min — Fichier : `exercices/exercice-04.md`

TP 4 : Étendre un volume logique et créer un volume BTRFS avec snapshot

- Durée : 1h30 — Fichier : `tp/tp-04.md`

Récap du module

- LVM : PV → VG → LV, redimensionnable à chaud
- Snapshot LVM = CoW bloc à taille fixe ; thin pool : alerter à 80 %
- BTRFS : CoW natif, subvolumes, snapshots quasi gratuits, compression
- Un snapshot n'est pas un backup : `btrfs send/receive` vers un autre device
- Choix : ext4 par défaut, XFS pour gros volumes parallèles, BTRFS pour snapshots fréquents

Cheatsheet : `cheatsheet/module-03.md`

Questions

Module suivant : Séquence d'amorçage et gestion de l'activité

Séquence d'amorçage et gestion de l'activité

Du firmware à l'invite de login, et la traçabilité qui suit

Linux - Administration avancée — M2I Formation
Module 4/6

Objectifs du module

À la fin de ce module, vous serez capable de :

- Détailler la séquence d'amorçage Linux moderne
- Passer des arguments au boot, reconstruire le boot, mesurer le démarrage
- Analyser les journaux systemd-journald et configurer la rétention

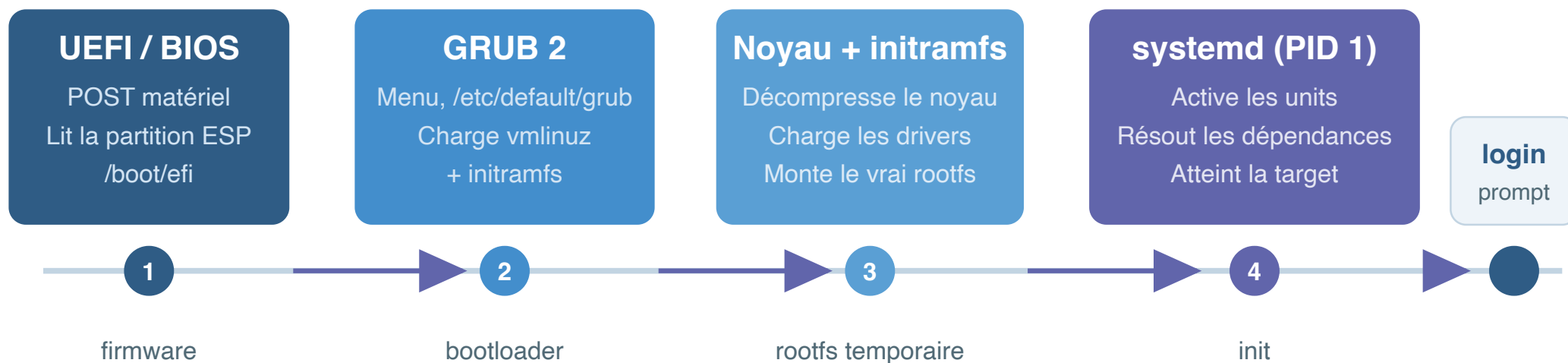
Durée : 3h30

Plan

1. Séquence d'amorçage et modes spéciaux
2. Reconstruction du boot et reset root
3. Gestion de l'activité : journald, rsyslogd, systemctl
4. Démo et TP
5. Récap et questions

1. Séquence d'amorçage

Vue d'ensemble du boot



Où intervenir quand ça casse

GRUB : touche « e » → éditer la ligne linux → ajouter systemd.unit=rescue.target ou init=/bin/bash

Les étapes en une ligne chacune

Étape	Rôle	Si ça casse ici
UEFI / BIOS	POST matériel, lit l'ESP	écran noir, pas de menu
GRUB 2	charge <code>vmlinux</code> + <code>initramfs</code>	<code>grub rescue></code>
initramfs	drivers, monte le vrai rootfs	prompt <code>(initramfs)</code>
systemd	active les units jusqu'à la target	boot lent, service en échec
login	invite utilisateur	système opérationnel

BIOS vs UEFI

Aspect	BIOS legacy	UEFI
Partition table	MBR	GPT
Bootloader	secteur 0 (MBR)	partition EFI (FAT32)
Secure Boot	non	oui
Taille disque max	2 To	9.4 ZB
Variables persistantes	non	efivars

```
[ -d /sys/firmware/efi ] && echo "UEFI" || echo "BIOS"
sudo efibootmgr -v
```

La partition système EFI (ESP)

- Partition **FAT32** de type EFI, montée sur `/boot/efi`
- Contient les bootloaders : `EFI/ubuntu/`, `EFI/BOOT/`
- Le firmware garde ses entrées de boot en NVRAM

```
ls /boot/efi/EFI/  
sudo efibootmgr -v          # entrées et ordre de boot  
sudo efibootmgr -o 0001,0000 # réordonner
```

Sur vos VM x86_64 : `grubx64.efi` (cible `x86_64-efi`). Sur la VM de démo ARM du formateur : `grubaa64.efi` (`arm64-efi`).

systemd-boot : l'alternative à GRUB

Bootloader minimaliste, UEFI uniquement, géré par `bootctl` :

```
# /boot/efi/loader/entries/ubuntu.conf
title      Ubuntu 24.04
linux      /vmlinuz
initrd     /initrd.img
options    root=UUID=... rw
```

- Pas de config générée : un fichier texte par entrée, dans l'ESP
- `bootctl install`, `bootctl status`, `bootctl list`
- GRUB reste le défaut Debian/Ubuntu ; systemd-boot équipe Pop!_OS, courant sur Arch

GRUB 2

Configuration générée à partir de fragments :

```
sudo nano /etc/default/grub      # TIMEOUT, GRUB_CMDLINE_LINUX_DEFAULT
ls /etc/grub.d/                  # scripts générateurs (10_linux, 40_custom)
sudo update-grub                 # Debian/Ubuntu -> /boot/grub/grub.cfg
sudo grub2-mkconfig -o /boot/grub2/grub.cfg # RHEL
```

- Ne jamais éditer `grub.cfg` à la main : régénéré à chaque MAJ du noyau
- Au menu : `e` pour éditer une entrée, `Ctrl+x` pour booter

Paramètres noyau utiles

Sur la ligne `linux` (touche `e`, ponctuel) ou dans `GRUB_CMDLINE_LINUX_DEFAULT` (persistant, puis `update-grub`) :

Paramètre	Usage
<code>nomodeset</code>	contourner un pilote vidéo (écran noir)
<code>net.ifnames=0</code>	retrouver les noms d'interfaces <code>eth0</code>
<code>crashkernel=256M</code>	réserver la RAM du noyau de capture (kdump)
<code>systemd.log_level=debug</code>	verbosité systemd au boot
retirer <code>quiet splash</code>	voir défiler les messages du boot

initramfs

Système de fichiers minimal en RAM, monté avant le rootfs :

- charge les modules nécessaires (LVM, RAID, chiffrement)
- monte le vrai rootfs
- passe la main au `/sbin/init` (systemd) du rootfs

Reconstruire :

```
sudo update-initramfs -u -k all          # Debian/Ubuntu, tous les noyaux
sudo dracut -f                          # RHEL/Fedora
ls /boot/initrd*
```

Que contient l'initramfs ?

```
lsinitramfs /boot/initrd.img-$(uname -r) | wc -l  
lsinitramfs /boot/initrd.img-$(uname -r) | grep lvm | head -4
```

Dedans : busybox, modules stockage et réseau, hooks LVM / LUKS / RAID, scripts de montage du rootfs.

À reconstruire après :

- ajout de LVM, LUKS ou RAID sous la racine
- changement de contrôleur ou de driver de disque
- modification des hooks `/etc/initramfs-tools/` (Debian/Ubuntu)

Le prompt (initramfs) : dépanner

Le shell BusyBox apparaît quand le vrai rootfs est introuvable : UUID changé, module manquant, FS corrompu.

```
(initramfs) blkid  
(initramfs) fsck /dev/sda2  
(initramfs) mount /dev/sda2 /root  
(initramfs) exit      # reprend le boot si /root est monté
```

Si `exit` échoue : noter l'erreur exacte, puis live USB et chroot (section 2).

systemd : targets et boot

Target	Équivalent runlevel
poweroff.target	0
rescue.target	1
multi-user.target	3
graphical.target	5
reboot.target	6
emergency.target	mode strict

```
systemctl get-default
sudo systemctl set-default multi-user.target
systemctl list-units --type=target
```

Modes spéciaux au boot

À l'écran GRUB, taper `e`, ajouter en fin de ligne `linux` :

Argument	Effet
<code>systemd.unit=rescue.target</code>	rescue (mono-user, FS montés)
<code>systemd.unit=emergency.target</code>	emergency (rootfs en RO, services off)
<code>single</code> ou <code>1</code>	équivalent rescue (compat)
<code>init=/bin/bash</code>	shell direct, sans systemd
<code>systemd.debug-shell=1</code>	shell sur tty9

2. Reconstruction du boot et reset root

Réinitialiser le mot de passe root

Au GRUB :

1. Taper `e` sur l'entrée
2. Ajouter `init=/bin/bash` à la ligne `linux`
3. `Ctrl+x` pour booter
4. Remonter `/` en RW :

```
mount -o remount,rw /  
passwd  
sync  
exec /sbin/init
```

Réparer un boot cassé

Symptôme courant : "no bootable device" ou kernel panic après MAJ.

```
# depuis un live USB
sudo mount /dev/sdaX /mnt
sudo mount --bind /dev /mnt/dev
sudo mount --bind /proc /mnt/proc
sudo mount --bind /sys /mnt/sys
sudo chroot /mnt
update-grub
update-initramfs -u
exit
sudo umount -R /mnt
```

kdump : capturer un kernel panic

Un panic ne laisse **aucune trace** : le noyau meurt avant d'écrire les journaux.

kdump réserve de la RAM pour un second noyau de capture :

```
sudo apt install kdump-tools      # ajoute crashkernel= à GRUB
kdump-config show                  # "ready to kdump" après reboot
grep crashkernel /proc/cmdline
```

Au panic : le noyau de capture démarre et copie la mémoire dans
`/var/crash/`.

Lire un vmcore

```
ls /var/crash/  
# 202608221030/dmesg.202608221030    <- premier réflexe  
# 202608221030/dump.202608221030    <- image mémoire  
  
less /var/crash/*/dmesg.*
```

- Le fichier `dmesg` suffit dans 90 % des cas : dernière pile d'appels, module fautif
- Analyse profonde : outil `crash` + symboles de débogage du noyau
- Test volontaire (VM uniquement) :
`echo c | sudo tee /proc/sysrq-trigger`

Mesurer le temps de démarrage

```
systemd-analyze  
systemd-analyze blame           # par service  
systemd-analyze critical-chain  # chemin critique  
systemd-analyze plot > boot.svg # diagramme
```

Cibles d'optimisation typiques :

- services activés inutilement
- timeouts réseau au boot
- dépendances mal déclarées

3. Gestion de l'activité

journald – collecteur unifié

`systemd-journald` collecte :

- messages noyau (kmsg)
- sortie standard et erreur des services
- syslog applicatif
- audit kernel

Stockage par défaut :

- `/run/log/journal/` (volatil) ou
- `/var/log/journal/` (persistant)

Activer la persistance :

Le flux des journaux

```

kmsg | stdout/stderr des units | /dev/log (syslog) | audit
      |
      | journald
      / \
/run/log/journal (volatil) /var/log/journal (persistant)
      |
ForwardToSyslog=yes -> rsyslogd -> syslog, auth.log
  
```

L'existence de `/var/log/journal/` est l'aiguillage de `Storage=auto` :
répertoire présent = journaux persistants.

journalctl — interroger les journaux

```
journalctl -b          # depuis le boot courant
journalctl -b -1       # boot précédent
journalctl -k          # noyau seulement
journalctl -u nginx    # un service
journalctl -p warning..err # par priorité
journalctl --since "1 hour ago" --until now
journalctl -f          # suivi en direct
journalctl --disk-usage
```

Priorités syslog et filtrage

Niveau	Nom	Émis par
0-2	emerg, alert, crit	pannes majeures
3	err	erreurs applicatives
4	warning	avertissements
5-7	notice, info, debug	fonctionnement normal, verbeux

- `-p err` = err **et plus grave** (0 à 3)
- Plage : `-p warning..err` (un second `-p` isolé écraserait le premier)
- Piège : une unit en échec émet en 4-5 → `-p err` ne montre rien, viser `-p warning`

journalctl : filtres par champs

Chaque entrée porte des champs indexés :

```
journalctl _SYSTEMD_UNIT=myapp.service
journalctl _PID=1234
journalctl _COMM=sshd --since today
journalctl -u myapp -o json | head -1      # export structuré
journalctl -o verbose -n 1                 # voir tous les champs
```

`-u myapp` est un raccourci qui combine `_SYSTEMD_UNIT=` avec les messages de systemd **sur** l'unité.

Configurer journald

/etc/systemd/journald.conf :

```
[Journal]
Storage=persistent
SystemMaxUse=2G
SystemKeepFree=500M
MaxRetentionSec=1month
ForwardToSyslog=yes
```

```
sudo systemctl restart systemd-journald
journalctl --vacuum-time=7d
journalctl --vacuum-size=500M
```

Rétro-compatibilité avec rsyslog

Beaucoup de chaînes attendent `/var/log/messages`, `/var/log/secure`.

`rsyslogd` peut consommer le journal et écrire dans des fichiers texte :

```
sudo apt install rsyslog
sudo systemctl enable --now rsyslog
# Debian/Ubuntu : syslog + auth.log – RHEL/Rocky : messages + secure
ls /var/log/{syslog,auth.log} 2>/dev/null || ls /var/log/{messages,secure}
```

Configuration : `/etc/rsyslog.conf` et `/etc/rsyslog.d/*.conf`.

systemctl — pilotage des services

```
systemctl status nginx
sudo systemctl start nginx
sudo systemctl enable --now nginx          # démarre + active au boot
sudo systemctl restart nginx
sudo systemctl reload nginx                # SIGHUP si supporté

systemctl is-failed nginx
systemctl list-units --failed
sudo systemctl reset-failed nginx

systemctl cat nginx                       # voir l'unit
sudo systemctl edit nginx                  # override propre
```

Anatomie d'une unit service

```
[Unit]
Description=Application de test LUX-ADMA
After=network.target
[Service]
ExecStart=/usr/local/bin/myapp.sh
Restart=on-failure
[Install]
WantedBy=multi-user.target
```

[Unit] : identité et dépendances — [Service] : quoi lancer et comment — [Install] : rattachement à la target (enable).

Type= et Restart=

Type=

Type	Comportement
simple	ExecStart EST le service
forking	le process se daemonise
oneshot	tâche qui se termine
notify	signale lui-même « prêt »

Restart=

Valeur	Relance si
no	jamais
on-failure	exit non nul, signal
always	toute fin

RestartSec= : délai entre deux essais.

Type=simple + script qui se termine = unité inactive : d'où le while true du TP 6.

StartLimitBurst : le garde-fou anti-boucle

Par défaut : **5** démarrages max par fenêtre de **10 s** (`StartLimitBurst=5`, `StartLimitIntervalSec=10s`).

- Tant que systemd retente : état `activating` (auto-restart)
- Limite atteinte → l'unité passe `failed`, plus aucune relance
- Repartir : `systemctl reset-failed myapp` puis `start`

Arithmétique du TP 6 : `RestartSec=1` → 5 essais en ~5 s, dans la fenêtre → `failed` observable. À 3 s, 15 s > 10 s : l'unité retenterait sans fin.

Modifier une unit proprement : les drop-ins

Ne jamais éditer une unit sous `/usr/lib/systemd/system/` : écrasée à la prochaine mise à jour du paquet.

```
sudo systemctl edit myapp          # crée l'override
# -> /etc/systemd/system/myapp.service.d/override.conf
sudo systemctl edit --full myapp    # copie complète dans /etc
sudo systemctl daemon-reload
systemctl cat myapp                # unit + drop-ins fusionnés
```

Un drop-in ne contient que les directives modifiées.

Diagnostiquer un service en échec

Boucle type :

```
systemctl status myapp          # voir le code de sortie
journalctl -u myapp -n 100      # logs récents du service
journalctl -u myapp -p warning  # jamais -p err : rien ne sort
systemctl cat myapp             # config réelle
sudo systemd-analyze verify /etc/systemd/system/myapp.service
```

Penser à `ExecStartPre` et aux dépendances `After=` / `Requires=`.

Démo live

Démo 7 : modes rescue / emergency / argument noyau

Démo 8 : reset du mot de passe root via GRUB

Démo 9 : journalctl sur un service en échec

Voir `demos/demo-07.md`, `demos/demo-08.md`, `demos/demo-09.md`

Travaux à suivre

Exercice 5 : Interpréter une sortie `journalctl` et identifier l'origine d'un échec

- `exercices/exercice-05.md` — 30 min

TPs

- TP 5 : Amorçage avancé (`tp/tp-05.md`) — 1h
- TP 6 : Configurer `journald` et diagnostiquer un service (`tp/tp-06.md`) — 1h

Récap du module

- BIOS/UEFI → bootloader → noyau + initramfs → systemd PID 1 → target
- Modes rescue/emergency/ `init=/bin/bash` permettent de récupérer un système
- `systemd-analyze` mesure et trace le boot
- journald centralise tous les journaux ; `journalctl` est l'outil pivot
- Diagnostic d'un service : `status` puis `journalctl -u` (`-p warning`, pas `-p err`)
- `Restart=` relance, `StartLimitBurst` protège : `reset-failed` pour

Questions

Module suivant : Maintenance système et configuration réseau

Maintenance système et configuration réseau

**Réagir vite, configurer juste, documenter
pour la prochaine fois**

Linux - Administration avancée — M2I Formation
Module 5/6

Objectifs du module

À la fin de ce module, vous serez capable de :

- Réagir en situation d'urgence (crash, FS corrompu, kernel panic)
- Maintenir la configuration réseau d'un serveur (NetworkManager, networkd, nmcli, ip)
- Mettre en place une procédure de maintenance reproductible

Durée : 3h30

Plan

1. Gestion d'urgence en cas de crash
2. Maintenance des applications
3. Maintenance de la configuration réseau
4. Démo et TP
5. Récap et questions

1. Gestion d'urgence en cas de crash

Symptômes typiques

Symptôme	Origine probable
Kernel panic au boot	initramfs cassé, FS root absent
FS read-only	erreur disque, corruption
Boucle de redémarrage	service essentiel qui crash
"No bootable device"	bootloader perdu, partition perdue
Mot de passe root oublié	reset par GRUB (cf. module 4)
Filesystem corrompu	crash dur, alimentation, RAID

Boîte à outils minimum

- Live USB de la distribution (ou Tails / SystemRescue)
 - Procédure de chroot écrite et testée
 - Sauvegarde du `/etc` et de `/boot`
 - Inventaire des partitions : `lsblk`, `blkid` exporté ailleurs
 - Accès console hors-bande (iLO, iDRAC, IPMI) sur serveurs
- “ Sans console, un serveur planté loin = déplacement physique. ”

Recovery par chroot

```
# depuis un live USB
sudo lsblk -f
sudo cryptsetup open /dev/sda3 root # si LUKS
sudo vgchange -ay # si LVM
sudo mount /dev/mapper/vg-root /mnt
sudo mount /dev/sda1 /mnt/boot/efi
sudo mount --bind /dev /mnt/dev
sudo mount --bind /proc /mnt/proc
sudo mount --bind /sys /mnt/sys
sudo chroot /mnt
```

Pourquoi les bind-mounts ?

Un chroot nu n'a ni périphériques ni vue du noyau :

- `/dev` : accès aux disques (`/dev/vda`, `/dev/mapper/...`)
- `/proc` : `update-grub` lit `/proc/mounts` pour trouver la racine
- `/sys` : variables EFI, indispensables à `grub-install` en UEFI
- `/run` : résolution DNS, LVM (souvent utile aussi)

Sans eux, `update-grub` et `grub-install` échouent avec des erreurs trompeuses. Cible : `x86_64-efi` sur vos VM (`arm64-efi` sur la démo du formateur).

Sortir du chroot proprement

Démontage en ordre **inverse** du montage :

```
exit                # quitter le chroot
sudo umount /mnt/sys /mnt/proc /mnt/dev
sudo umount /mnt/boot/efi
sudo umount /mnt
sudo reboot
```

Un `umount /mnt` direct échoue (`target is busy`) tant que les sous-montages existent. Raccourci : `umount -R /mnt` démonte récursivement.

fsck : les règles du jeu

- **Jamais sur un FS monté en lecture-écriture** : corruption garantie
- Depuis un live USB, ou sur une partition non-root démontée
- Le FS racine se vérifie au boot : ajouter `fsck.mode=force` à la ligne noyau

Codes retour de `fsck` (cumulables) :

Code	Signification
0	aucune erreur
1	erreurs corrigées
2	corrigées, redémarrage requis
4	erreurs restantes non corrigées

Réparer un FS

Le FS doit être démonté.

```
# depuis un live USB
sudo umount /dev/sda3
sudo fsck -y /dev/sda3           # ext*
sudo xfs_repair /dev/sda3       # XFS (mode R0 si log dirty)
sudo btrfs check --readonly /dev/sda3
```

Sur ext4 dont le journal est sain, `fsck` au boot suffit (forcer avec `fsck.mode=force` dans GRUB).

e2fsck et xfs_repair : options utiles

```
sudo e2fsck -p /dev/vda3      # "preen" : ne corrige que ce qui est sûr
sudo e2fsck -y /dev/vda3      # répond oui à tout (réparations lourdes)
sudo xfs_repair -n /dev/vdb1   # dry-run : liste sans rien modifier
sudo xfs_repair /dev/vdb1      # répare (exige un journal propre)
sudo xfs_repair -L /dev/vdb1   # vide le journal : DERNIER recours
```

`xfs_repair -L` détruit les transactions non rejouées : tenter d'abord un montage/démontage pour rejouer le journal.

Kernel panic : que noter ?

- Stack trace : nom du module en haut
- Message exact ("not syncing", "Unable to handle kernel paging request")
- Photo de l'écran ou journaux netconsole
- Dernier changement (kernel update, driver, hardware)

```
# activer un crashdump (kdump)
sudo apt install kdump-tools
sudo systemctl enable --now kdump-tools
ls /var/crash/
```

2. Maintenance des applications

Gestion des paquets

```
# Debian/Ubuntu  
sudo apt update  
sudo apt list --upgradable  
sudo apt full-upgrade  
sudo apt autoremove --purge  
apt-cache policy nginx
```

```
# RHEL/Rocky  
sudo dnf check-update  
sudo dnf upgrade  
sudo dnf history list  
sudo dnf history undo <id>
```

Toujours lire la liste des paquets retirés avant de valider.

Sources et signatures

```
ls /etc/apt/sources.list.d/  
ls /etc/apt/keyrings/  
ls /etc/yum.repos.d/  
  
# Debian moderne  
cat /etc/apt/sources.list.d/myrepo.sources
```

Vérifier qu'un dépôt tiers signe ses paquets et que la clé GPG est en `keyrings/`.

Geler et prioriser des paquets

```
sudo apt-mark hold nginx          # bloquer toute mise à jour  
apt-mark showhold  
sudo apt-mark unhold nginx
```

Pinning fin dans `/etc/apt/preferences.d/nginx.pref` :

```
Package: nginx  
Pin: version 1.24.*  
Pin-Priority: 1001
```

Mises à jour automatiques et traçabilité

```
sudo apt install unattended-upgrades  
sudo dpkg-reconfigure -p low unattended-upgrades  
sudo unattended-upgrade --dry-run
```

- Config : `/etc/apt/apt.conf.d/50unattended-upgrades` (sécurité seule par défaut)
- Historique APT : `/var/log/apt/history.log` (quoi, quand, quelle commande)
- Détail par paquet : `/var/log/dpkg.log`

Cycle de vie d'un service

```
sudo systemctl daemon-reload      # après édition d'unit
sudo systemctl enable --now myapp
sudo systemctl reload myapp        # SIGHUP, sans coupure
sudo systemctl restart myapp
sudo systemctl mask myapp          # interdire toute activation
sudo systemctl unmask myapp
```

`systemctl edit myapp` crée un override propre dans
`/etc/systemd/system/myapp.service.d/override.conf`.

Snapshots de configuration

Avant un changement risqué :

```
sudo cp /etc/nginx/nginx.conf{,.bak}  
sudo etckeeper commit "before nginx tuning" # si etckeeper
```

etckeeper versionne /etc dans Git automatiquement après chaque apt.

```
sudo apt install etckeeper  
sudo etckeeper init
```

3. Maintenance réseau

Outils modernes vs legacy

Tâche	Moderne	Legacy
Lister interfaces	<code>ip link</code>	<code>ifconfig</code>
Adresse IP	<code>ip addr</code>	<code>ifconfig</code>
Routes	<code>ip route</code>	<code>route -n</code>
ARP / neighbors	<code>ip neigh</code>	<code>arp -a</code>
Sockets	<code>ss -tulpn</code>	<code>netstat -tulpn</code>

`ip` et `ss` sont fournis par le paquet `iproute2`. Utiliser ces commandes par défaut.

Inspection rapide

```
ip -br addr          # vue compacte
ip -br link
ip route
ip -s link show eth0 # compteurs RX/TX et erreurs
ss -tulpn            # ports en écoute + processus
ss -s                # résumé sockets
```

NetworkManager (nmcli)

Sur stations de travail et serveurs RHEL :

```
nmcli device status
nmcli connection show
sudo nmcli connection add type ethernet con-name ens3 ifname ens3 \
    ipv4.method manual ipv4.addresses 192.168.1.50/24 \
    ipv4.gateway 192.168.1.1 ipv4.dns 1.1.1.1
sudo nmcli connection up ens3
```

Configuration en `/etc/NetworkManager/system-connections/`.

systemd-networkd

Sur serveurs minimalistes (cloud, Debian sans NM) :

```
# /etc/systemd/network/10-eth0.network
```

```
[Match]
```

```
Name=ens3
```

```
[Network]
```

```
Address=192.168.1.50/24
```

```
Gateway=192.168.1.1
```

```
DNS=1.1.1.1
```

```
sudo systemctl enable --now systemd-networkd systemd-resolved  
networkctl status
```

nmcli ↔ networkd : correspondances

Geste	nmcli	systemd-networkd
État des interfaces	<code>nmcli device status</code>	<code>networkctl list</code>
Détail d'une interface	<code>nmcli device show ens3</code>	<code>networkctl status ens3</code>
IP statique	<code>nmcli con add ... ipv4.addresses</code>	fichier <code>.network</code>
Appliquer	<code>nmcli con up ens3</code>	<code>networkctl reload</code>
Fichiers	<code>system-connections/</code>	<code>/etc/systemd/network/</code>

Un seul gestionnaire par interface : la colonne **SETUP** de `networkctl list` dit qui pilote quoi.

.netdev vs .network : deux rôles distincts

Fichier	Rôle	Exemples
.netdev	Créer un device virtuel	bond, VLAN, bridge
.network	Configurer un device existant	IP, routes, DNS, rattachement

- Une interface physique n'a **jamais** de .netdev : elle existe déjà
- Un bond exige les **deux** : un .netdev (le créer) + un .network (l'adresser)
- Fichiers lus par ordre lexical : préfixer d'un numéro (10-, 11-...)

Exemple : un bond en trois fichiers

```
# 10-bond0.netdev
[NetDev]
Name=bond0
Kind=bond

[Bond]
Mode=active-backup
MIIMonitorSec=0.1
```

```
# 11-enp7s0.network
[Match]
Name=enp7s0

[Network]
Bond=bond0
```

```
# 12-bond0.network
[Match]
Name=bond0

[Network]
Address=10.0.42.10/24
```

Règle d'or : l'interface de secours

Avant toute manipulation réseau sur une machine distante :

1. Garder une voie d'accès **indépendante** : console, IPMI, ou une NIC dédiée hors bond/VLAN
2. Tester cette voie **avant** de toucher au reste
3. Ne jamais reconfigurer l'interface qui porte la session SSH en cours
4. Programmer un filet : `sudo shutdown -r +10`, annulé par `shutdown -c` une fois la config validée

Le TP 8 garde `enp1s0` hors du bond : c'est ce filet.

Modes de bonding : lequel choisir ?

Mode	Tolérance panne	Débit agrégé	Prérequis switch
active-backup	oui	non	aucun
802.3ad (LACP)	oui	oui	LACP configuré
balance-rr	oui	oui (désordre TCP)	ports agrégés
balance-xor	oui	par flux	ports agrégés

Sans switch LACP (VM, lab), 802.3ad ne négocie **aucun** agrégat : le mode démontrable est active-backup.

VLAN, bonding, bridge

```
# VLAN 42 sur eth0
sudo ip link add link eth0 name eth0.42 type vlan id 42
sudo ip addr add 10.0.42.10/24 dev eth0.42
sudo ip link set eth0.42 up

# bonding (mode sans prérequis switch)
sudo modprobe bonding
sudo ip link add bond0 type bond mode active-backup miimon 100
sudo ip link set eth0 master bond0
sudo ip link set eth1 master bond0
```

À rendre persistant via `nmcli`, `systemd-networkd` ou `/etc/network/interfaces`.

Bonding et VLAN empilés

SERVICES

Administration

SSH, supervision

Production

application métier

VLAN

802.1Q

bond0.42

VLAN 42 — 10.0.42.10/24

bond0.100

VLAN 100 — 10.0.100.10/24

Un tag 802.1Q par trame :
un seul lien physique,
plusieurs réseaux logiques.

AGRÉGAT

bond0

mode 802.3ad (LACP) · miimon 100 · lacp_rate fast

INTERFACES

enp7s0

slave — pas d'IP

enp8s0

slave — pas d'IP

enp1s0

accès de secours

Toujours garder une voie
d'accès hors du bond.

SWITCH — port channel LACP — trunk VLAN 42, 100

systemd-resolved : l'architecture

- Stub DNS local en écoute sur `127.0.0.53:53`
- `/etc/resolv.conf` est un lien vers `stub-resolv.conf`
- Les serveurs réels sont attachés **par interface** (via `.network` ou `nmcli`)
- Cache intégré, LLMNR et mDNS optionnels

```
resolvectl status          # serveurs réels, par interface
resolvectl query srv1.lan
resolvectl flush-caches
```

Pièges de /etc/resolv.conf

- C'est un **lien symbolique** : une édition manuelle est écrasée ou ignorée
- `nameserver 127.0.0.53` = resolved actif ; les vrais serveurs sont dans `resolvectl status`
- Un outil qui lit resolv.conf directement (chroot, conteneur) ne voit que le stub
- DNS fixe : le déclarer via `DNS=` dans le `.network`, ou `nmcli`, jamais à la main

```
ls -l /etc/resolv.conf  
resolvectl dns
```

DNS et résolution

```
resolvectl status           # systemd-resolved  
resolvectl query example.com  
cat /etc/resolv.conf        # peut être un lien  
dig +short example.com @1.1.1.1  
getent hosts example.com    # passe par NSS
```

nsswitch.conf arbitre l'ordre files dns mdns etc.

Démo live

Démo 10 : recovery d'un système non bootable via live USB et chroot

Démo 11 : configuration réseau persistante avec nmcli puis systemd-networkd

Voir `demos/demo-10.md` et `demos/demo-11.md`

Travaux à suivre

Exercice 6 : Rédiger une procédure de recovery pour un serveur en panne

- `exercices/exercice-06.md` — 30 min

TPs

- TP 7 : Recovery d'un système Linux (`tp/tp-07.md`) — 1h
- TP 8 : Configurer une interface, un bonding et un VLAN (`tp/tp-08.md`) — 1h

Récap du module

- En cas de crash : live USB + chroot, puis fsck/xfs_repair/btrfs check
 - Paquets : utiliser le gestionnaire natif, vérifier les sources signées
 - `etckeeper` versionne `/etc` automatiquement
 - Réseau : `ip`, `ss`, et `nmcli` ou `systemd-networkd` pour la persistance
 - Toujours documenter les étapes pour reproduire la procédure
- Cheatsheet : `cheatsheet/module-05.md`

Questions

Module suivant : Performances et sécurité

Performances et sécurité

Mesurer, identifier les goulots, durcir

Linux - Administration avancée – M2I Formation

Module 6/6

Objectifs du module

À la fin de ce module, vous serez capable de :

- Analyser les performances CPU, mémoire, disque et réseau
- Identifier les goulots d'étranglement et proposer des résolutions
- Mettre en oeuvre des bonnes pratiques de durcissement et de sécurité

Durée : 3h30

Plan

1. Méthode d'analyse des performances
2. Outils par couche (CPU, mémoire, disque, réseau)
3. Bonnes pratiques de durcissement
4. Sécurité de l'identité, du réseau, des données
5. Démo et TP
6. Récap et questions

1. Méthode d'analyse

Approche systématique

Méthode "USE" (Utilization, Saturation, Errors) de Brendan Gregg :

- **Utilization** : pourcentage de temps occupé
- **Saturation** : files d'attente, blocages
- **Errors** : erreurs visibles dans les compteurs

À appliquer à chaque ressource : CPU, mémoire, disque, réseau.

“ "If you can't measure it, you can't improve it." — Lord Kelvin ”

USE appliquée aux quatre ressources

Méthode USE — Brendan Gregg

Pour CHAQUE ressource : Utilization · Saturation · Errors

% de temps occupé →

Utilization

Saturation

Errors

CPU

mpstat · top · pidstat

%us + %sy

mpstat -P ALL 1

load > nb cores

uptime · runqueue

throttling

dmesg · thermal

Mémoire

free · vmstat

available

pas « free » !

swap si/so

vmstat 1

OOM killer

dmesg -T

Disque

iostat -xz 1

%util

~100 % = saturé

await · aqu-sz

file d'attente

erreurs I/O

smartctl · dmesg

Approcher un problème de perf

1. Cadrer : symptôme exact, depuis quand, périmètre
2. Mesurer **avant** de modifier (baseline)
3. Identifier la couche limitante
4. Une seule modification à la fois
5. Mesurer à nouveau
6. Documenter

Sans baseline, toute optimisation est une rumeur.

2. Outils par couche

CPU — vue rapide

```
top                # ou htop
mpstat -P ALL 1    # par CPU
pidstat 1          # par processus
uptime            # load average
```

Lectures :

- `load average` : voir les deux slides suivantes (pas seulement du CPU)
- `%us` élevé → code applicatif
- `%sy` élevé → kernel/syscalls
- `%wa` élevé → attente I/O

Lire le load average

```
$ uptime
```

```
17:02:11 up 12 days, 3 users, load average: 6.12, 5.80, 4.95
```

- Trois moyennes : 1, 5 et 15 minutes → lire la **tendance**
- Compte les processus prêts à courir (état R) **et bloqués en I/O (état D)**
- Repère : comparer au nombre de coeurs (`nproc`)
- 6.0 sur 8 coeurs : de la marge ; 6.0 sur 2 coeurs : saturation

Le piège n°1 : load haut, CPU inoccupé

Observation	Conclusion
load 8, %idle proche de 0	CPU réellement saturé
load 8, %idle 90 %, %wa haut	goulot disque , pas CPU
load 8, %idle 90 %, %wa bas	processus en D (NFS bloqué, driver...)

```
ps -eo state,pid,comm | awk '$1=="D"' # qui est bloqué en I/O ?
```

Ajouter des coeurs à une machine qui attend son disque ne change rien.

perf : disponibilité réelle

- Paquet Ubuntu : `linux-tools-generic` (et `linux-tools-$(uname -r)`)
- En VM, les compteurs matériels manquent souvent : `perf` retombe sur des événements logiciels (précision moindre, mais utilisable)
- `kernel.perf_event_paranoid` restreint l'usage non-root

```
sudo apt install linux-tools-generic  
sysctl kernel.perf_event_paranoid
```

Si `perf` n'est pas disponible sur votre lab : les commandes qui suivent sont à connaître comme culture générale.

CPU — profiling

```
sudo perf top  
sudo perf record -F 99 -a -g -- sleep 30  
sudo perf report
```

Pour générer un flamegraph :

```
git clone https://github.com/brendangregg/FlameGraph  
sudo perf record -F 99 -a -g -- sleep 30  
sudo perf script | FlameGraph/stackcollapse-perf.pl | FlameGraph/flamegraph.pl > flame.svg
```

Mémoire

```
free -h  
vmstat 1  
cat /proc/meminfo  
ps aux --sort=-rss | head  
slabtop
```

Indicateurs :

- `available` (pas `free`) reflète la mémoire utilisable
- `swap si/so` régulier → pression mémoire
- OOM killer dans `dmesg`

MemAvailable vs MemFree

```
grep -E 'MemFree|MemAvailable' /proc/meminfo
```

- **MemFree** : pages strictement inutilisées — **toujours bas** sur un serveur sain
- Le cache de pages occupe la RAM libre : c'est voulu, il se libère à la demande
- **MemAvailable** : estimation de ce qu'une application peut obtenir **sans swapper**
- Alerter sur **MemAvailable**, jamais sur **MemFree**

Pression mémoire et OOM killer

- Vraie pression : `si/so` non nuls **en continu** (`vmstat 1`) et `MemAvailable` qui tend vers zéro
- À court de mémoire, le noyau tue le processus au **score OOM** le plus haut

```
sudo dmesg | grep -i "out of memory"
cat /proc/$(pgrep -x myapp)/oom_score
echo -500 | sudo tee /proc/$(pgrep -x myapp)/oom_score_adj
```

Trace complète dans `journalctl -k` : victime, score, état mémoire au moment du kill.

Disque

```
iostat -xz 1          # %util, await, r/s, w/s
sudo iotop            # par processus (ou iotop-c, plus moderne)
sudo blktrace -d /dev/sda
sar -d 1 5           # historique avec sysstat
```

Goulots typiques :

- `%util` ~ 100% sur un device
- `await` élevé → latence disque
- I/O scheduler inadapté (`mq-deadline`, `none`, `bfq`)

Réseau

```
ss -s                # résumé sockets
ss -ti              # détail TCP par socket
ip -s link show eth0 # erreurs RX/TX, drops
sudo iftop          # par hôte
sudo nethogs        # par processus
mtr 8.8.8.8          # path + perte
nstat -az           # compteurs noyau
```

Erreurs RX dropped → MTU, buffers, file noyau (`net.core.rmem_max`).

Tester avec charge

```
# CPU
stress-ng --cpu $(nproc) --timeout 60s --metrics
sysbench cpu --threads=$(nproc) run

# disque
fio --name=randread --filename=/data/test --size=1G --bs=4k \
    --rw=randread --ioengine=libaio --iodepth=32 --runtime=30 --time_based

# réseau
iperf3 -s                # côté serveur
iperf3 -c srv -P 4        # côté client, 4 flux
```

Toujours sur un environnement isolé.

Goulots typiques et résolutions

Symptôme	Cause probable	Action
%wa élevé	I/O disque saturé	EBS plus rapide, scheduler, cache
Load haut, %us OK	CPU saturé	profiler, scaler
Swap actif	RAM insuffisante	ajouter RAM, réduire heap
RX dropped	net.core.rmem_max trop bas	augmenter via sysctl
Latence DB	requêtes manquant d'index	EXPLAIN, ajouter index

Supervision centralisée

Stack de référence :

- Métriques : Prometheus + node_exporter
- Logs : Loki (ou ELK)
- Visualisation : Grafana
- Alertes : Alertmanager

Sur l'hôte : exporter `node_exporter` qui expose `/metrics` sur `:9100`.

```
curl localhost:9100/metrics | head
```

3. Bonnes pratiques de durcissement

Principe — surface d'attaque minimale

- N'installer que ce qui est nécessaire
- Désactiver les services non utilisés
- Restreindre les comptes et privilèges
- Mettre à jour régulièrement
- Journaliser et auditer

Référence : CIS Benchmarks (Linux Distribution Independent), ANSSI guides.

Durcissement SSH

`/etc/ssh/sshd_config.d/99-hardening.conf` :

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
LoginGraceTime 30s
AllowUsers admin alice
ClientAliveInterval 300
ClientAliveCountMax 2
```

```
sudo sshd -t && sudo systemctl reload ssh
```

Comptes et sudo

```
# politiques mot de passe
sudo nano /etc/security/pwquality.conf

# expiration
sudo chage -M 90 -m 7 -W 14 alice
sudo chage -l alice

# sudo granulaire (jamais ALL=(ALL))
echo "alice ALL=(root) NOPASSWD: /usr/bin/systemctl reload nginx" \
    | sudo tee /etc/sudoers.d/alice
sudo visudo -c
```

sudoers : la dernière règle gagne

sudo évalue **toutes** les règles applicables et retient la **dernière** :

```
alice ALL=(root) NOPASSWD: /usr/bin/systemctl reload nginx
%admin ALL=(ALL:ALL) ALL
```

Si alice est membre de `%admin`, la seconde règle s'applique aussi : elle conserve tous les droits, la restriction est illusoire.

Ordre de lecture : `/etc/sudoers` puis `/etc/sudoers.d/` en ordre lexical.

Un `ALL` final annule silencieusement tout ce qui précède.

sudo ciblé : bonnes pratiques

- Une règle = un fichier dans `/etc/sudoers.d/`, nom sans point ni `~`
- `NOPASSWD` : uniquement sur des commandes **précises**, chemin absolu
- Pas de joker exploitable : `systemctl restart nginx`, jamais `systemctl *`
- `visudo -c` valide **la syntaxe seulement** — ni la logique, ni l'ordre des règles

```
sudo visudo -f /etc/sudoers.d/alice # édition avec verrou + contrôle
sudo -l -U alice                   # ce qu'alice peut vraiment faire
```

sysctl sécurité

```
# /etc/sysctl.d/99-hardening.conf
kernel.kptr_restrict=2
kernel.dmesg_restrict=1
kernel.unprivileged_bpf_disabled=2
net.ipv4.conf.all.rp_filter=1
net.ipv4.conf.all.accept_redirects=0
net.ipv4.conf.all.send_redirects=0
net.ipv4.tcp_syncookies=1
net.ipv6.conf.all.accept_ra=0
fs.protected_hardlinks=1
fs.protected_symlinks=1
```

```
sudo sysctl --system
```

Pare-feu

nftables (moderne) ou firewalld (RHEL) :

```
sudo nft list ruleset
# la table et la chaine doivent exister AVANT d'ajouter une regle
sudo nft add table inet filter
sudo nft 'add chain inet filter input { type filter hook input priority 0; policy drop; }'
sudo nft add rule inet filter input ct state established,related accept
sudo nft add rule inet filter input iif lo accept
sudo nft add rule inet filter input tcp dport 22 accept

# firewalld
sudo firewall-cmd --add-service=https --permanent
sudo firewall-cmd --reload
```

Toujours valider depuis une autre console avant de couper la connexion en cours.

nftables : la structure

Trois niveaux, à créer dans cet ordre :

1. **Table** : espace de nommage par famille (`inet` = IPv4 + IPv6)
2. **Chaîne** : point d'accrochage (`type filter hook input priority 0`) et **policy**
3. **Règle** : critère + verdict (`accept`, `drop`, `reject`)

```
sudo nft list ruleset  
sudo nft -c -f /etc/nftables.conf # -c : vérifie SANS appliquer
```

Persistance Ubuntu : `/etc/nftables.conf` + `systemctl enable nftables`

policy drop ou règle drop finale ?

policy drop

- Verdict par défaut de la chaîne
- Sûr : tout ce qui n'est pas accepté tombe
- S'applique **dès la création** de la chaîne : charger policy et règles accept dans le même fichier

Règle drop finale

- **policy accept** + un **drop** en dernière règle
- Un flush des règles laisse tout **ouvert**
- Acceptable en phase de mise au point seulement

Durcir sans se couper la main

1. Garder une **seconde session SSH ouverte** pendant toute l'opération
2. Valider la syntaxe avant d'appliquer : `sshd -t`, `visudo -c`, `nft -c -f`
3. Appliquer, puis tester avec une **nouvelle** connexion — la session en cours survit souvent à un pare-feu mal réglé (conntrack)
4. Prévoir le retour arrière : `sudo shutdown -r +10`, annulé par `shutdown -c` une fois validé
5. Ne fermer la session de secours qu'après le test complet

SELinux et AppArmor

MAC	Distros	Mode
SELinux	RHEL, Fedora	enforcing / permissive / disabled
AppArmor	Debian, Ubuntu, SUSE	par profil

```
getenforce
sudo aa-status
sudo ausearch -m AVC -ts recent      # SELinux denials
```

Ne pas désactiver. Investiguer les denials, écrire des modules ou des profils adaptés.

auditd : la boîte noire du noyau

`auditd` journalise les accès aux fichiers et appels système que vous désignez.

Règle de surveillance : `-w` chemin, `-p` opérations (r, w, x, a = attributs), `-k` clé de recherche :

```
-w /etc/passwd -p wa -k passwd_changes  
-w /etc/sudoers -p wa -k sudoers_changes
```

- Persistance : fichiers `/etc/audit/rules.d/*.rules`, puis `sudo augenrules --load`
- Règles actives : `sudo auditctl -l`

Exploiter l'audit : ausearch et aureport

```
sudo ausearch -k sudoers_changes      # événements d'une clé
sudo ausearch -m USER_LOGIN -ts today  # logins du jour
sudo aureport --summary                 # vue d'ensemble
sudo aureport -au                       # tentatives d'authentification
```

- Journal brut : `/var/log/audit/audit.log`
- Le TP 10 pose quatre règles `-w` : vérifiez-les avec `ausearch` après avoir modifié un fichier surveillé

4. Sécurité identité, réseau, données

Panorama — pistes à approfondir après la formation

Identité

- PAM pour la chaîne d'authentification
- SSSD ou nslcd pour l'annuaire (LDAP, AD, Kerberos)
- MFA via `pam_oath` ou `google-authenticator`
- Audit des comptes inactifs

```
lastlog | sort -k 5  
sudo passwd -l zombieuser  
sudo usermod -L -e 1 zombieuser
```

Réseau

- TLS partout (Let's Encrypt, internal CA)
- Cipher suites modernes uniquement
- HSTS, OCSP stapling, certificats courts
- Segmentation : zone privée/publique, VLAN
- WireGuard pour les tunnels site-à-site

```
openssl s_client -connect example.com:443 -servername example.com < /dev/null  
testssl.sh https://example.com
```

Données

- Chiffrement disque : LUKS au repos
- Chiffrement applicatif : champs sensibles
- Sauvegardes chiffrées (restic, borg)
- Rétention et test de restauration
- Effacement : `cryptsetup erase`, `shred` (sur disque magnétique)

```
sudo cryptsetup luksFormat /dev/sdb  
sudo cryptsetup open /dev/sdb data  
sudo mkfs.xfs /dev/mapper/data
```

Gestion des clés et chiffrement de flux

- Pour SSH : clés ed25519, passphrase, agent
- Pour TLS : ACME, certbot, lego, cert-manager
- Pour secrets applicatifs : Vault, sops, age
- Rotation automatisée et audit des accès

```
ssh-keygen -t ed25519 -C "alice@laptop"  
age-keygen -o key.txt  
echo "secret" | age -r $(grep -v '#' key.txt) -o secret.age
```

Démo live

Démo 12 : analyse complète CPU/mémoire/disque/réseau

Démo 13 : durcissement minimal d'un serveur (SSH, sudo, sysctl)

Voir `demos/demo-12.md` et `demos/demo-13.md`

Travaux à suivre

Exercice 7 : Interpréter un dossier de mesures et proposer un plan d'action

- `exercices/exercice-07.md` — 45 min

TPs

- TP 9 : Analyse de performances complète (`tp/tp-09.md`) — 1h
- TP 10 : Durcissement d'un serveur Linux (`tp/tp-10.md`) — 1h

Récap du module

- Méthode USE (Utilization, Saturation, Errors) appliquée à chaque ressource
- Toujours mesurer avant et après une modification
- Durcir = réduire la surface d'attaque + journaliser + tenir à jour
- SSH, sudo, sysctl, pare-feu, MAC, auditd : briques minimales du durcissement
- Durcir en séquence : session de secours ouverte, valider (-t, -c), puis fermer
- Identité, réseau, données : chacun a sa chaîne de chiffrement et d'audit

Questions et bilan

Merci pour ces 3 jours.

Restez en contact via myM2i – 30 jours d'accompagnement.